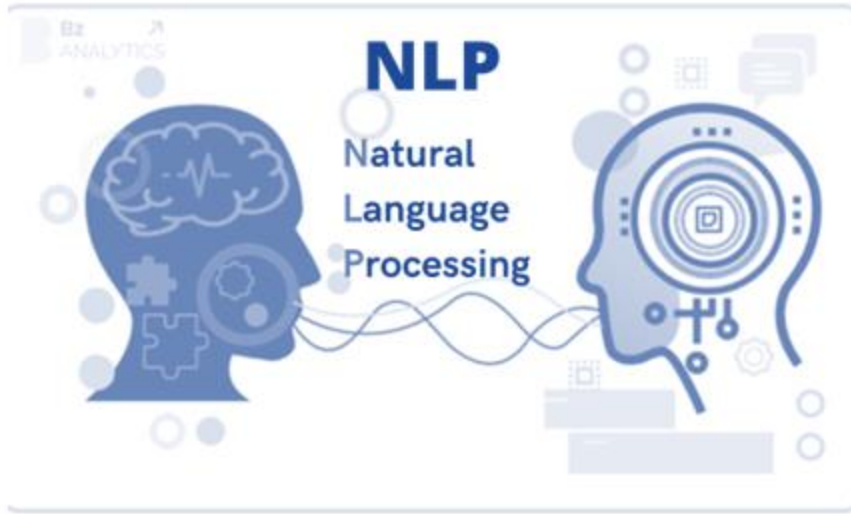




Natural language processing (NLP)

Session Four

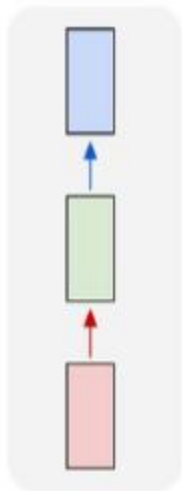
Outline:

1	RNN architectures
2	Sequence-to-sequence
3	Beam Search
4	Evaluation with BLEU
5	Bottleneck
6	Attention mechanism

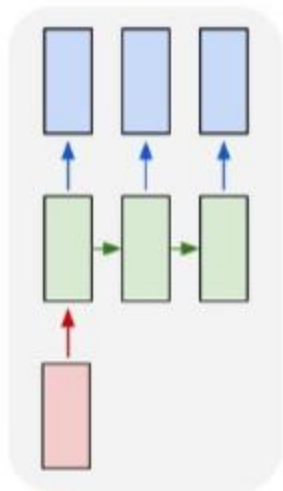
RNN architectures

Recall that there are **different ways to set up RNNs** to solve different kinds of problems.

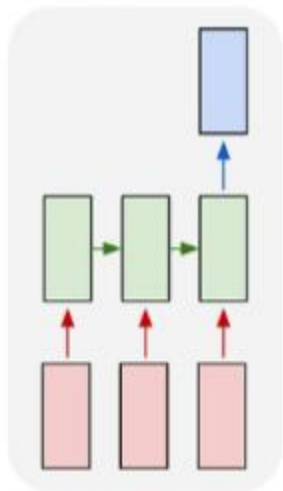
one to one



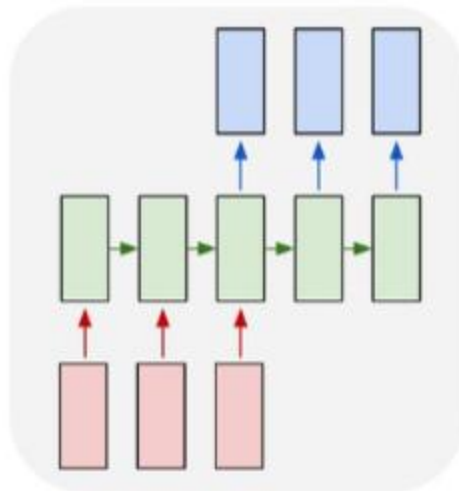
one to many



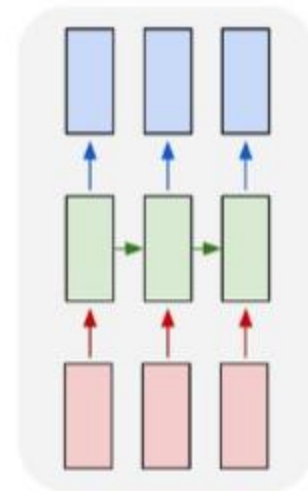
many to one



many to many



many to many



RNN architectures

Recall that there are **different ways to set up RNNs** to solve different kinds of problems.

one-to-one - is equivalent to a feed-forward neural network.

one-to-many - can be used for captioning, decoders or music generation.

many-to-one - is used for classification tasks like sentiment analysis.

many-to-many - used for sequence labelling and language modelling tasks.

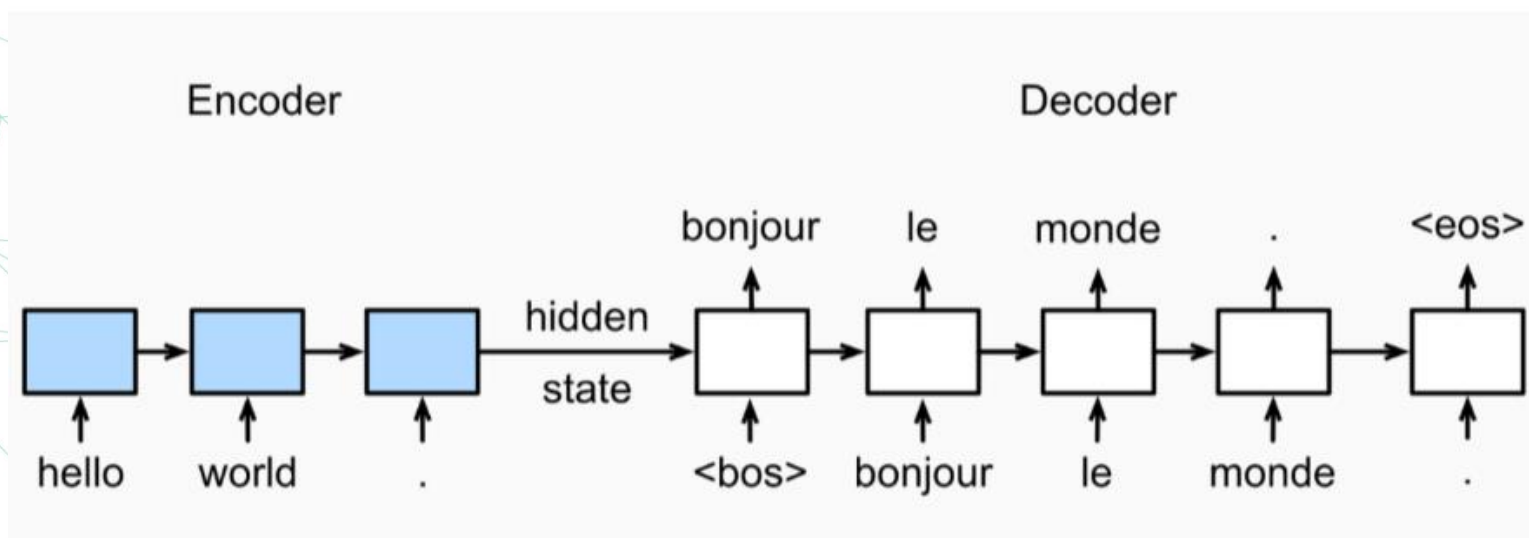
staggered many-to-many - (also known as sequence-to-sequence) its applications include translation, summarization and chatbots.

We're going to focus on the last type, i.e. **sequence-to-sequence** (aka seq2seq). This architecture is the most **flexible** because it can take in a sequence of one length and output a sequence of completely **different** length.

Sequence-to-sequence

The seq2seq architecture can be broken down into 2 sub models - the **encoder** and the **decoder**. The encoder takes in the input sequence and processes it whereas the decoder takes the output of the encoder and uses this to output a new sequence.

The RNN cells can be either plain, GRU or LSTM as this doesn't change the architecture



Sequence-to-sequence

- Observe how the intermediate outputs of the encoder are **not used** at all in this setup. The only thing that gets passed to the decoder is the **final hidden state**, which can be thought of as an **embedding** of the input sequence.
- The first step of the decoder takes in the encoder embedding and a beginning of sentence token. It uses this to generate an output that is passed through a **softmax function** to produce a **probability distribution** over the target language vocabulary. The most probable word is selected as the **prediction**, which is then fed in as the **input** for the **next time step** of the decoder and the process repeats.
- Because the decoder makes predictions based on the encoder output, this is considered a **conditional language model**. This is trained using a **self-supervised teacher forcing** approach. That is, we get it to **predict the next word** in the sequence given the correct previous words in the translation. **Backpropagation** is used to update the weights of the encoder and decoder simultaneously, i.e. it is treated as a **single system**.

A RNN Language Model

A RNN Language Model

output distribution

$$\hat{y} = \text{softmax}(W_2 h^{(t)} + b_2)$$

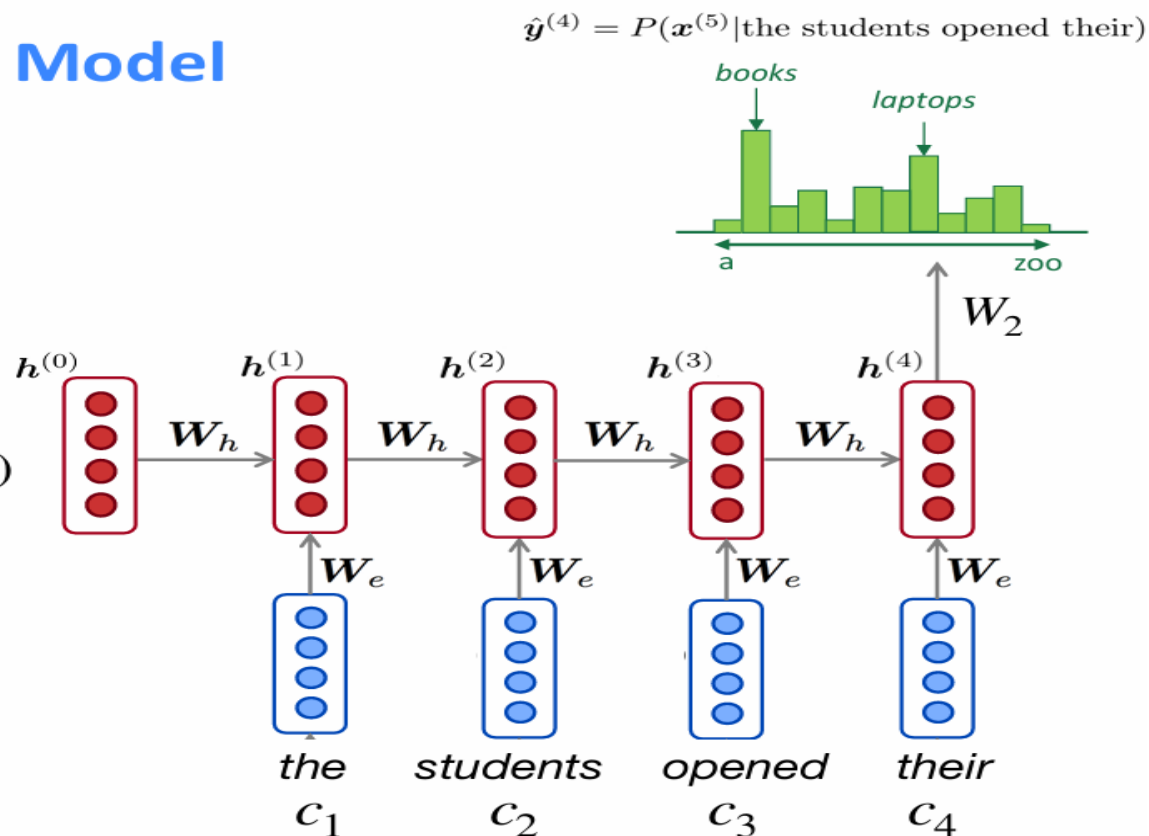
hidden states

$$h^{(t)} = f(W_h h^{(t-1)} + W_e c_t + b_1)$$

$h^{(0)}$ is initial hidden state!

word embeddings

c_1, c_2, c_3, c_4



A RNN Language Model

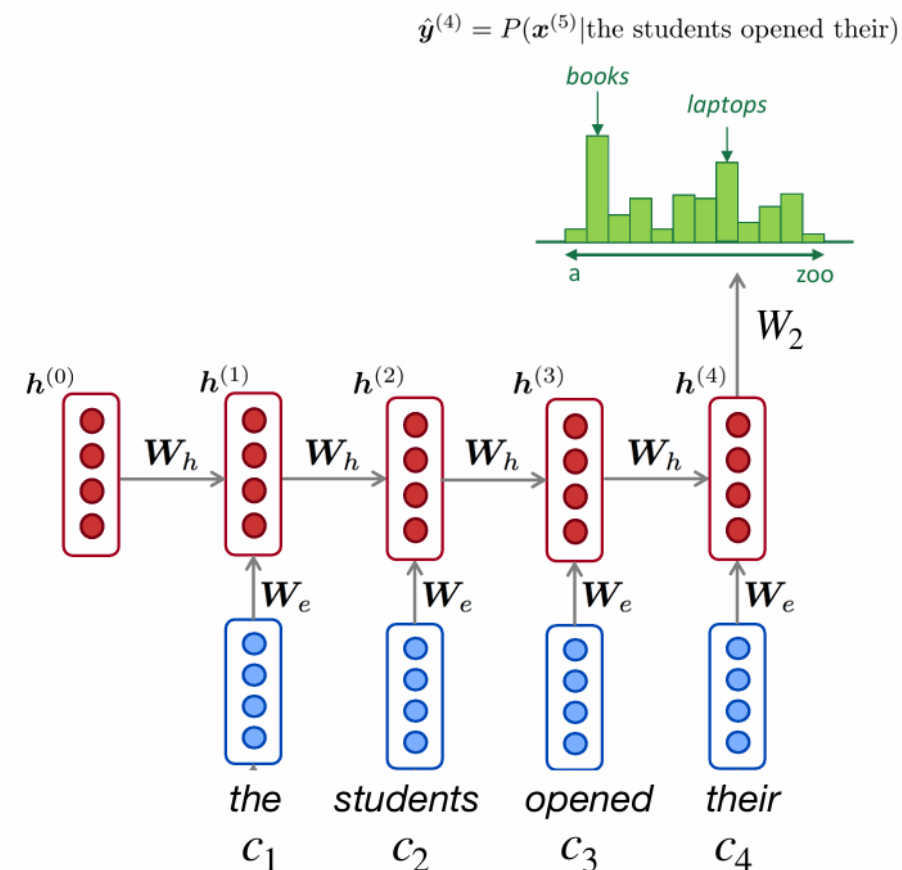
RNN Advantages:

- Can process any length input
- Model size doesn't increase for longer input
- Computation for step t can (in theory) use information from many steps back

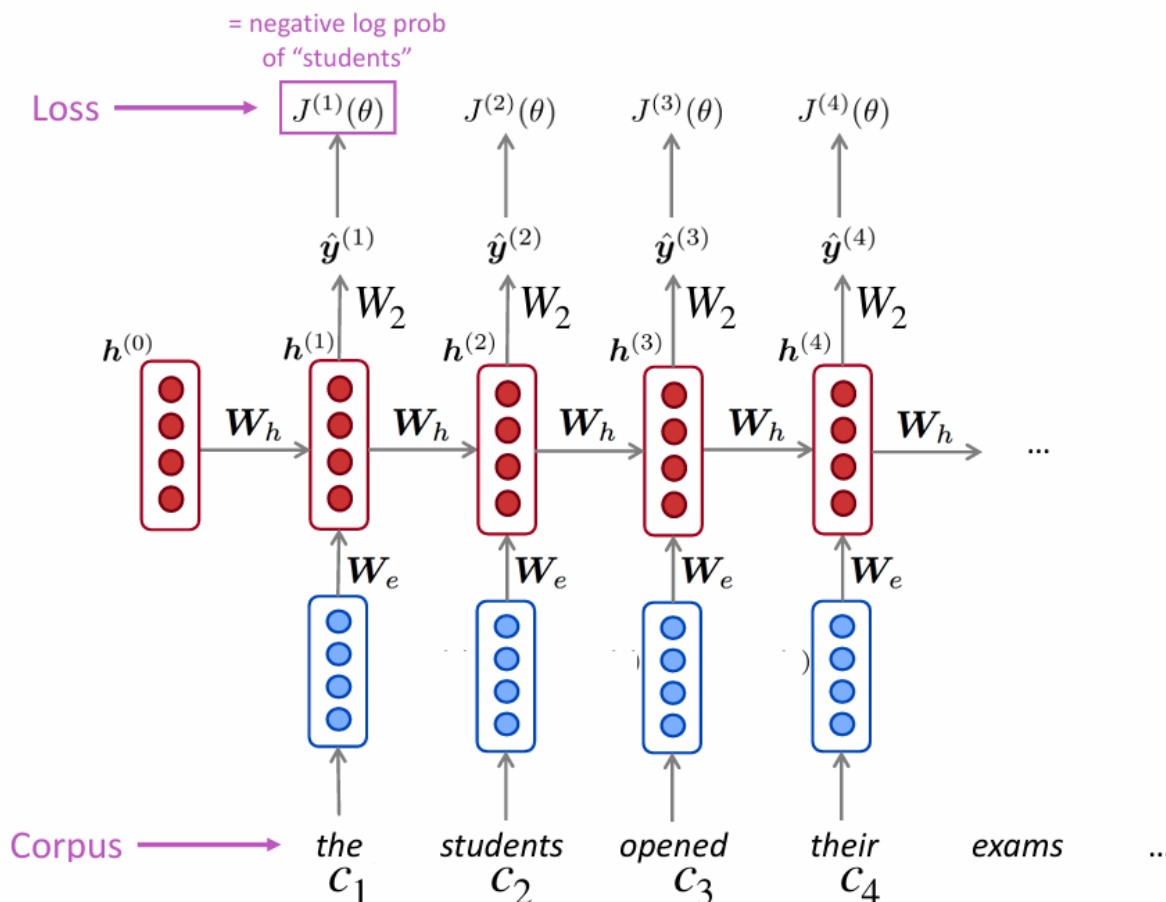
- Weights are shared across timesteps representations are shared

RNN Disadvantages:

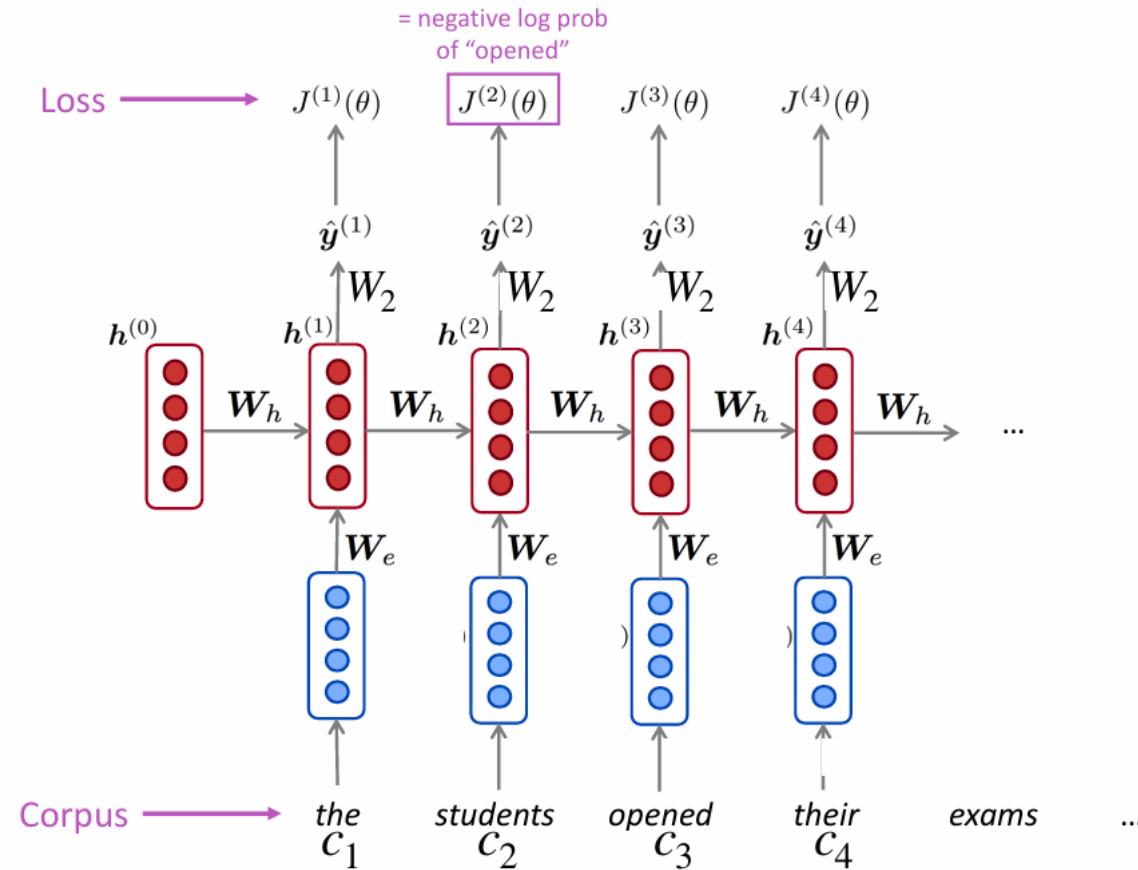
- Recurrent computation is slow
- In practice, difficult to access information from many steps back



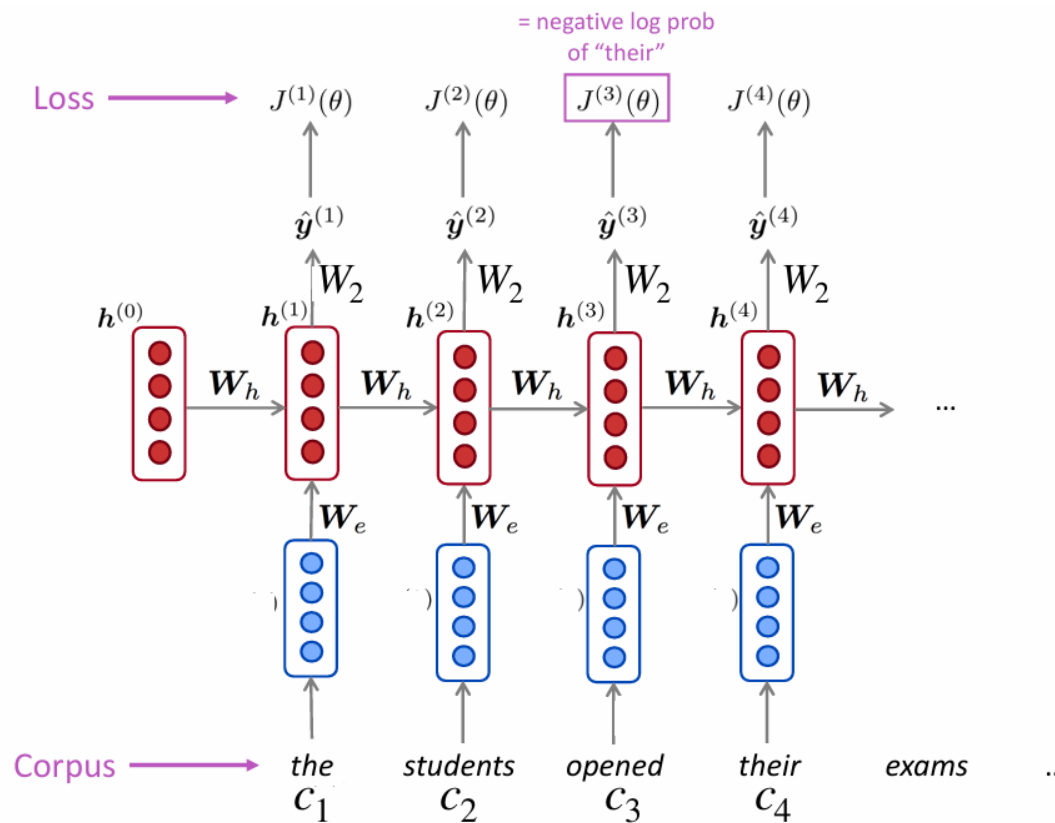
Training a RNN Language Model



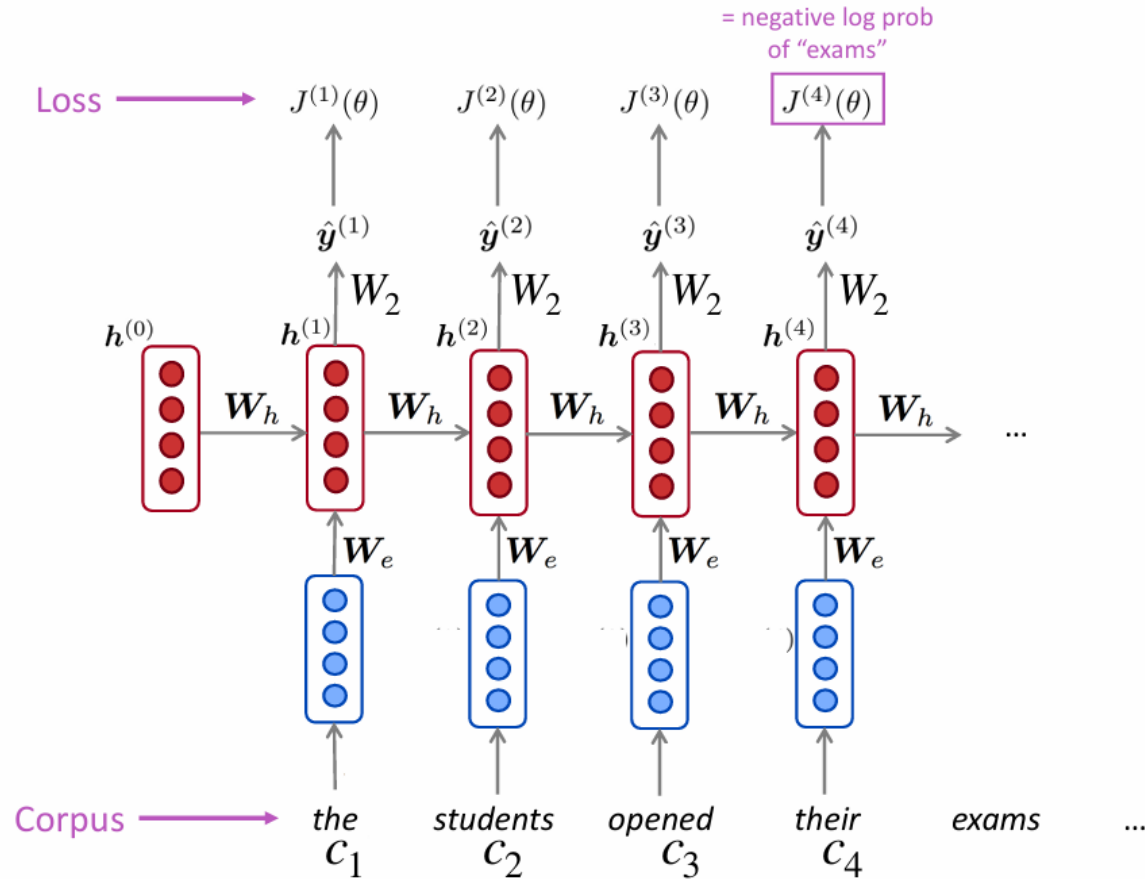
Training a RNN Language Model



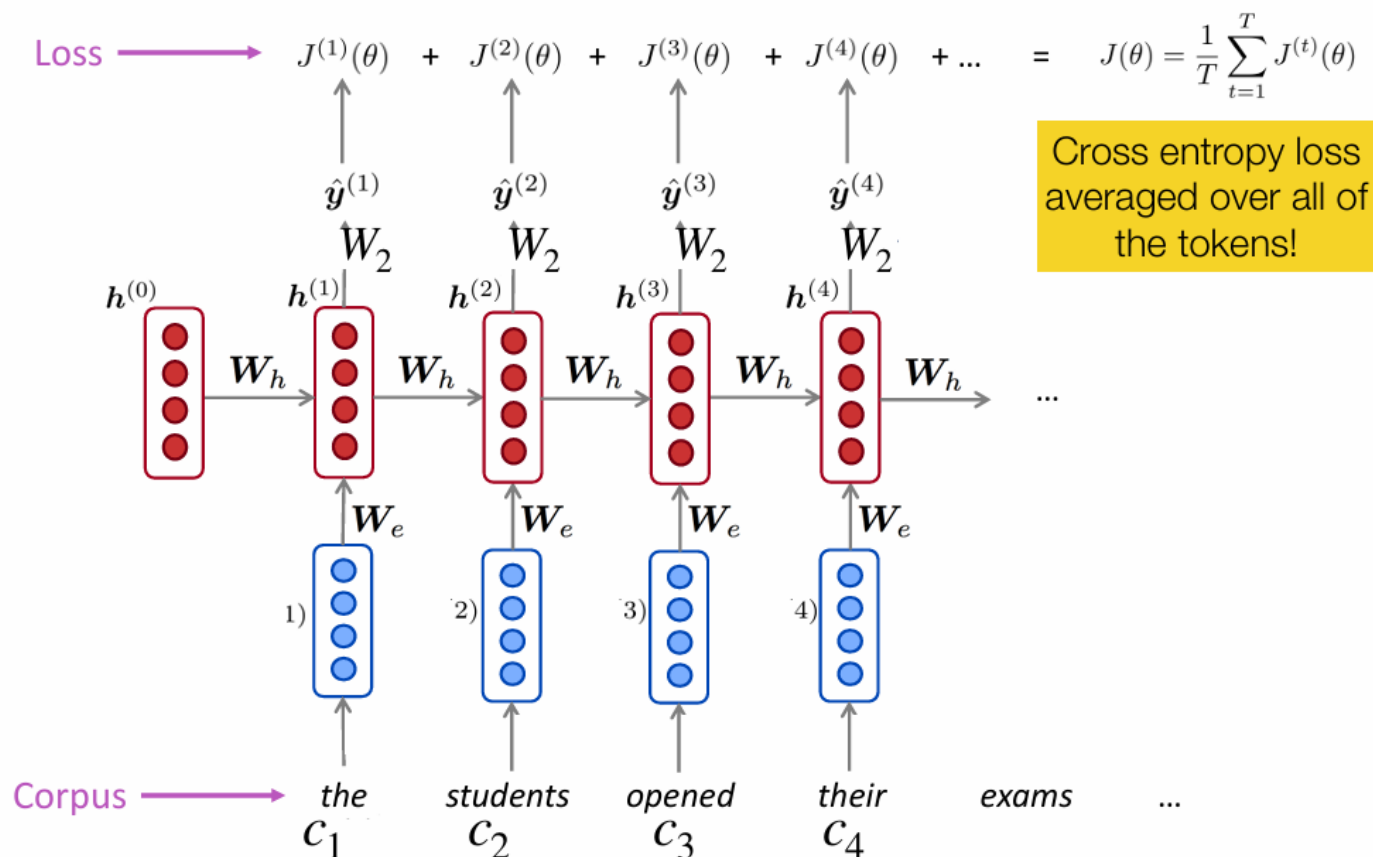
Training a RNN Language Model



Training a RNN Language Model



Training a RNN Language Model



Challenges

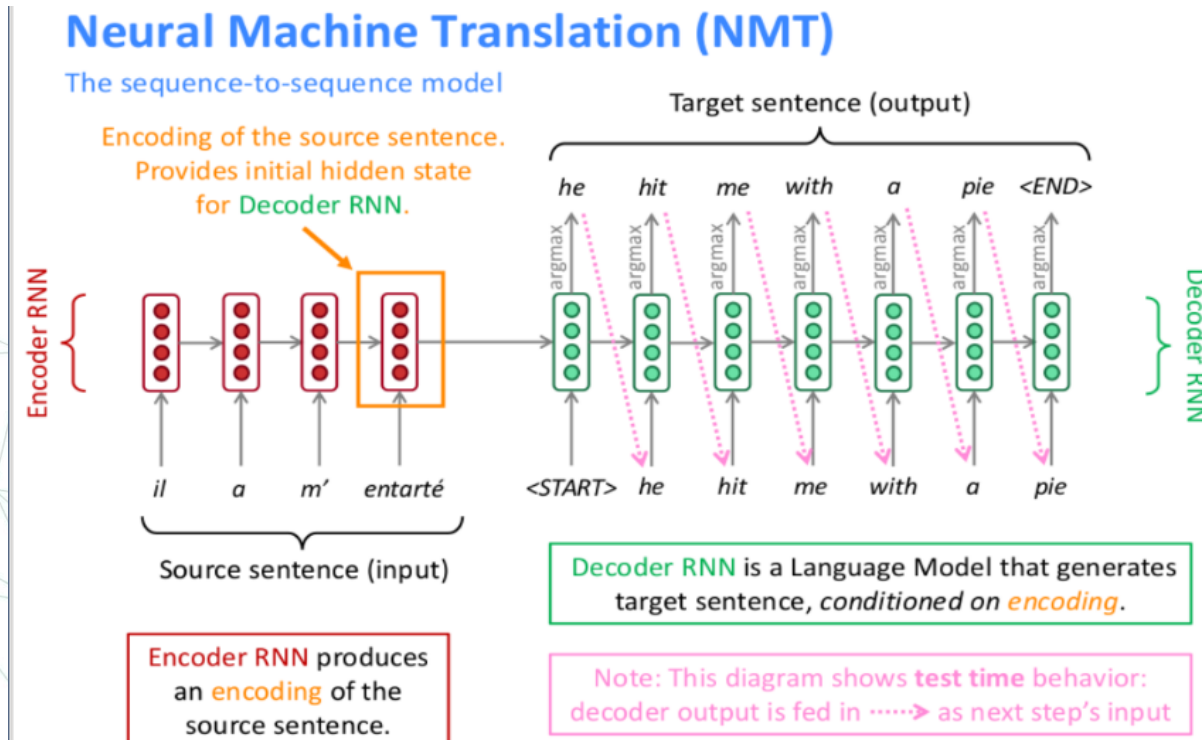
Machine Translation is a very **challenging problem** because different languages can express the same concept in different ways. In particular, some of the difficulties include

- **Word alignment** - a word in one language may be expressed as 2 words in another language and vice versa.
- **Word order structure** - English is a Subject-Verb-Object (SVO) language whereas Japanese for example is a Subject-Object-Verb (SOV) language. So "Joe ate cake" would be expressed as "Joe cake ate" in Japanese.
- **Adequacy vs fluency** - there are often many ways to express the same information in a language but some of them will sound more fluent and natural to a native speaker than others.
- **Bias** - all humans are biased in some way, therefore very large sets of diverse training data are needed to train accurate and unbiased translation models

What is Neural Machine Translation ?

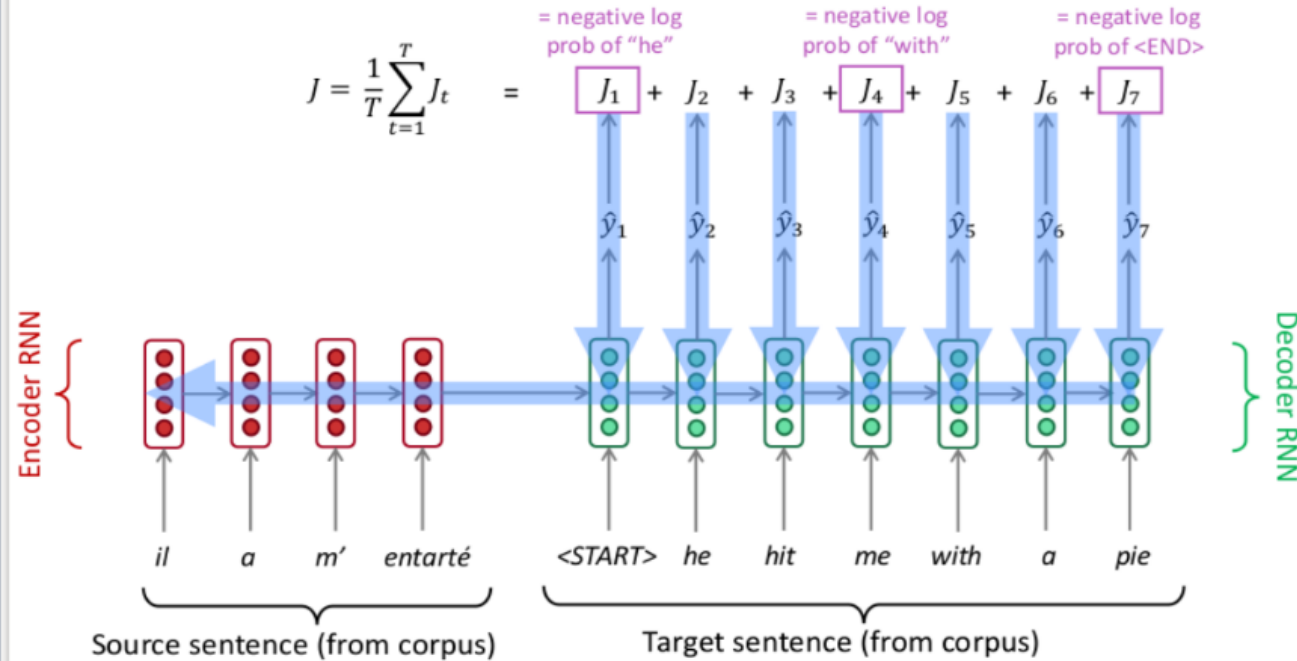
- Neural Machine Translation (NMT) is a way to do Machine Translation with single neural network
- The neural network architecture is called sequence-to sequence (aka seq2seq)

What is Neural Machine Translation ?



What is Neural Machine Translation ?

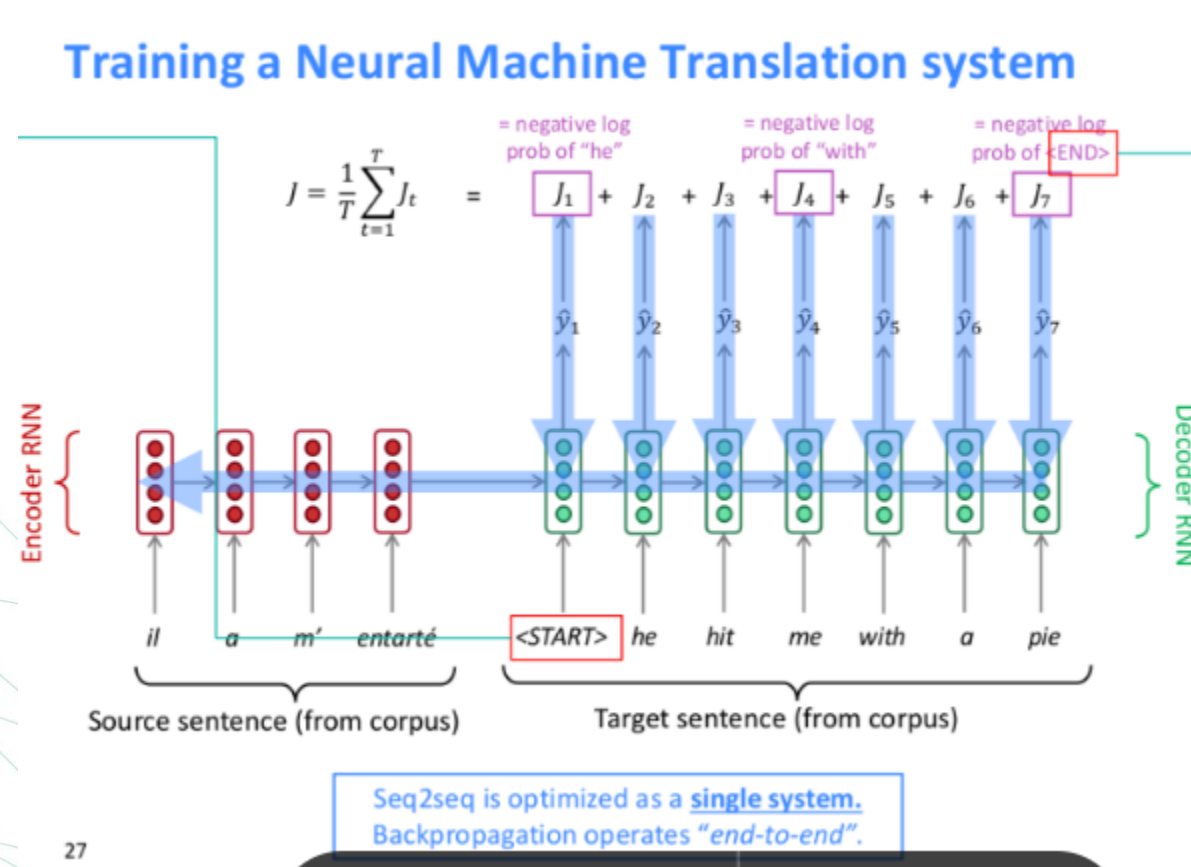
Training a Neural Machine Translation system



Seq2seq is optimized as a single system.
 Backpropagation operates "end-to-end".

What is Neural Machine Translation ?

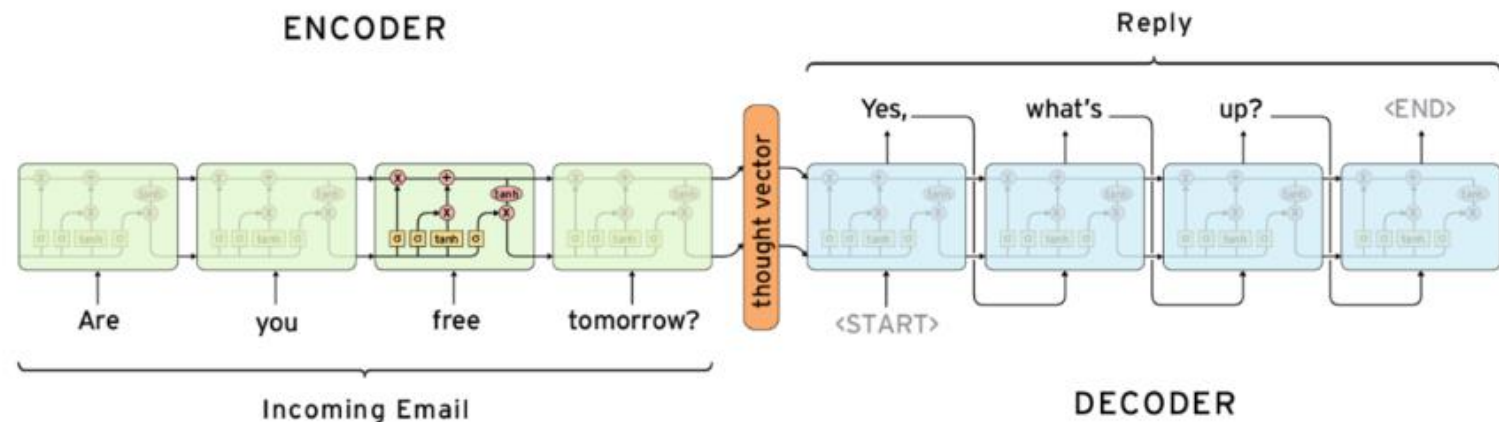
Notice the addition of **<START>** token to the beginning of target → To kick start the decoder



Notice the addition of **<END>** token to the end of target → To mark end of generation of Decoder.
More on that in Decoding

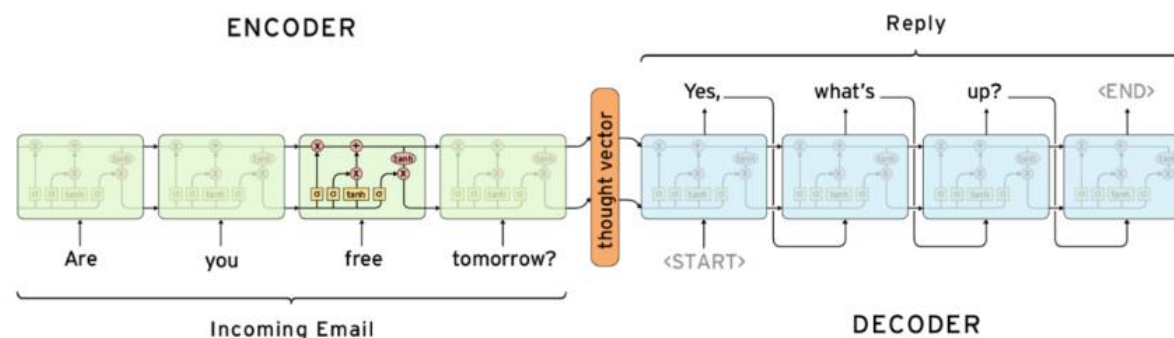
Training

- RNN has 2 inputs: X and $h(t-1)$. LSTM: $h=[c;h]$
- Encoder: the whole input seq is first fed $\rightarrow X=\text{true input}$
- Decoder: The encoder state = decoder init state. $X = ?$
 - - $X[0] = \langle \text{START} \rangle$
 - - Next $X[t] = y[t-1] \rightarrow \text{GT}$



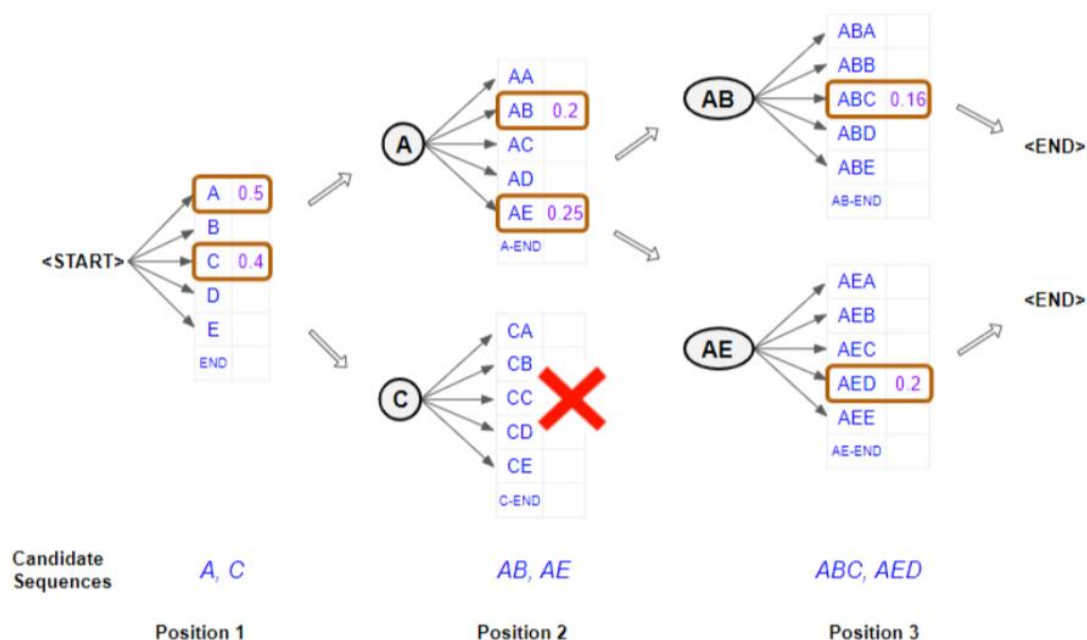
Inference

- RNN has 2 inputs: X and $h(t-1)$. LSTM: $h=[c;h]$
- Encoder: the whole input seq is first fed $\rightarrow X=\text{true input}$
- Decoder: The encoder state = decoder init state. $X = ?$
 - - $X[0] = \langle \text{START} \rangle$
 - - Next $X[t] = y[t-1] \rightarrow \text{Prediction}$
 - - **Stop on generating $\langle \text{END} \rangle$**



Beam Search

- We are currently using a **greedy decoding** strategy, i.e. the most **probable word** is chosen at each time step. But this doesn't necessarily produce the best translations. This is because we are looking for the **highest probability sentences**, and for example perhaps the second most probable word at the first time step would have led to a better scoring sentence overall because later words would have been a better match. Greedy search only explores a **narrow** set of all the possible combinations.



Beam Search

Beam search decoding

- Core idea: On each step of decoder, keep track of the *k most probable* partial translations (which we call *hypotheses*)
 - *k* is the *beam size* (in practice around 5 to 10)
- A hypothesis $y_1, \dots, y_t$ has a *score* which is its log probability:
$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$
 - Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top *k* on each step
- Beam search is *not guaranteed* to find optimal solution
- But *much more efficient* than exhaustive search!

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$

<START>

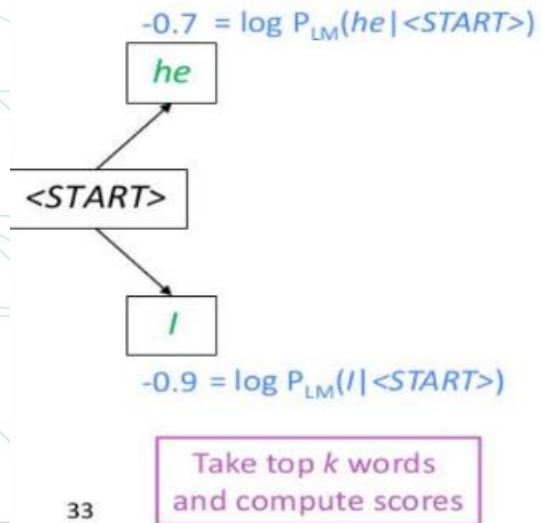
Calculate prob
dist of next word

32

Beam Search

Beam search decoding: example

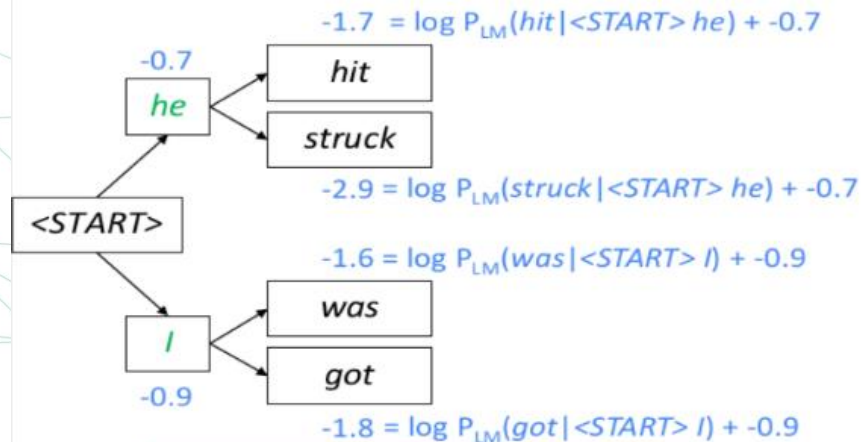
Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

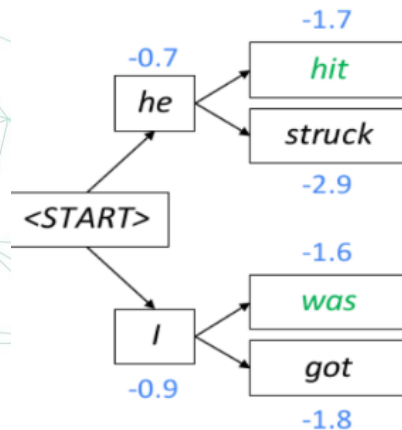


For each of the k hypotheses, find top k next words and calculate scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

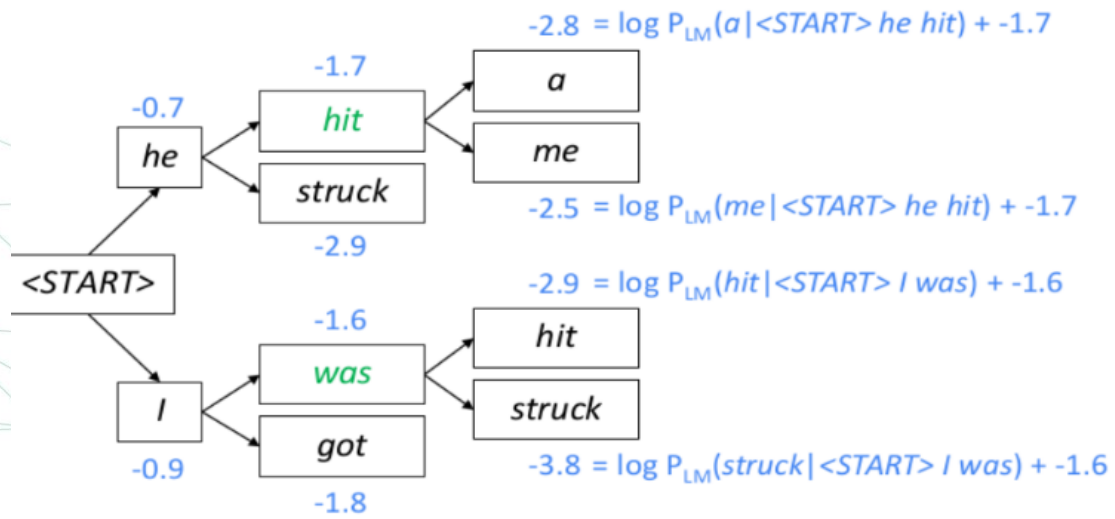


Of these k^2 hypotheses,
just keep k with highest scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

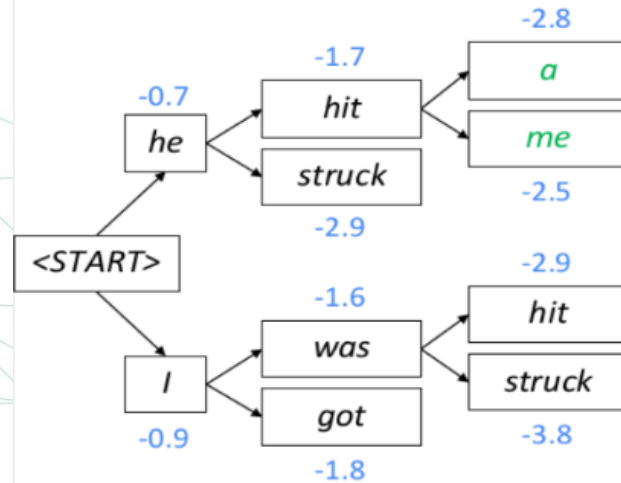


For each of the k hypotheses, find top k next words and calculate scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

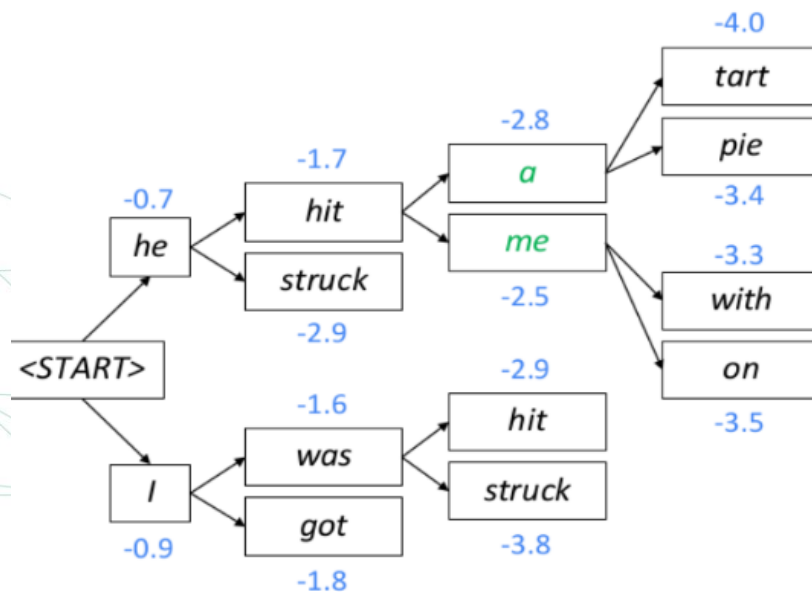


Of these k^2 hypotheses,
just keep k with highest scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

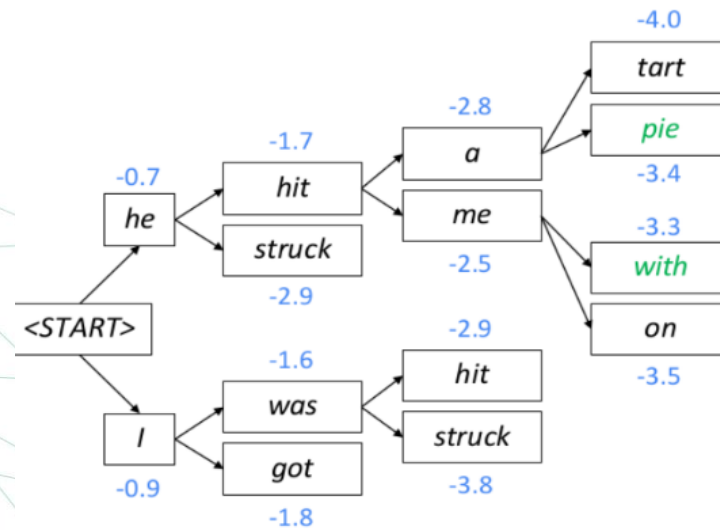


For each of the k hypotheses, find top k next words and calculate scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

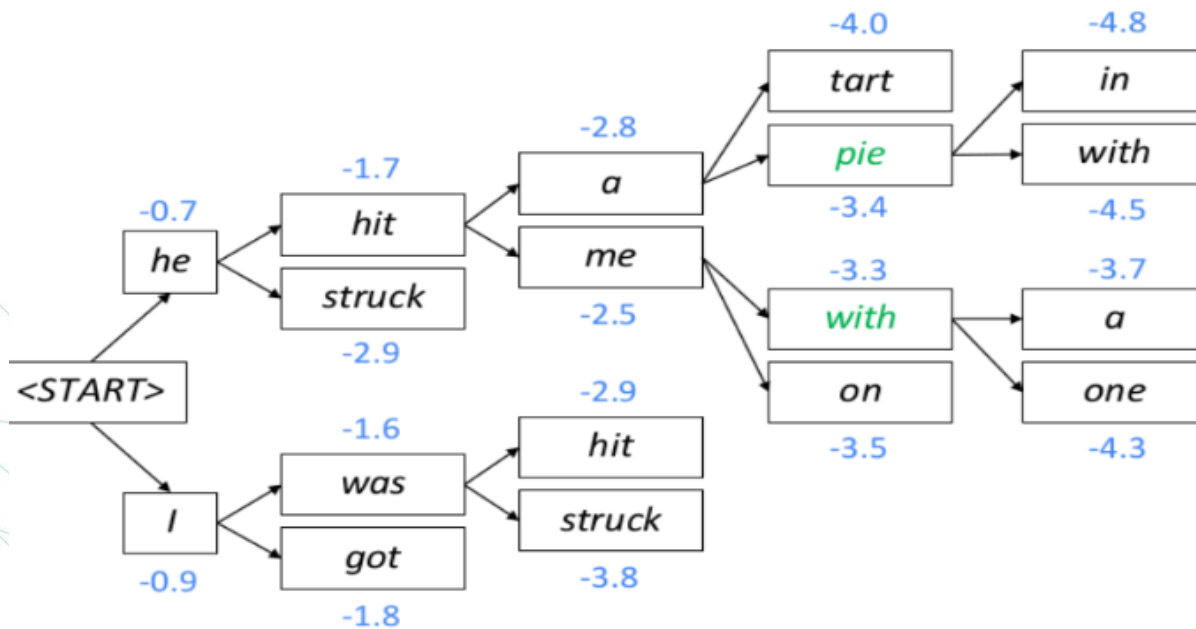


Of these k^2 hypotheses,
just keep k with highest scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

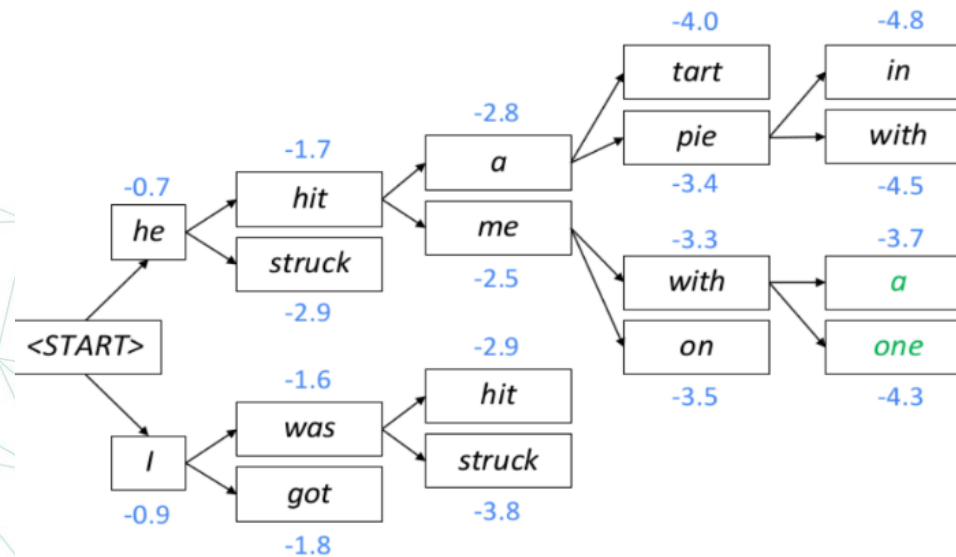


For each of the k hypotheses, find top k next words and calculate scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

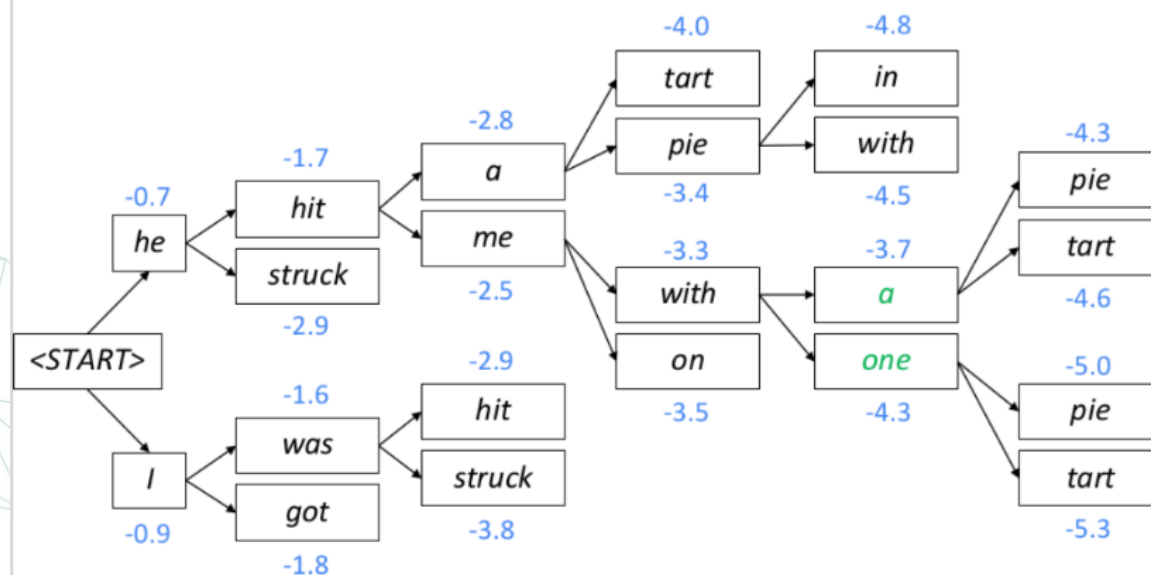


Of these k^2 hypotheses,
just keep k with highest scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

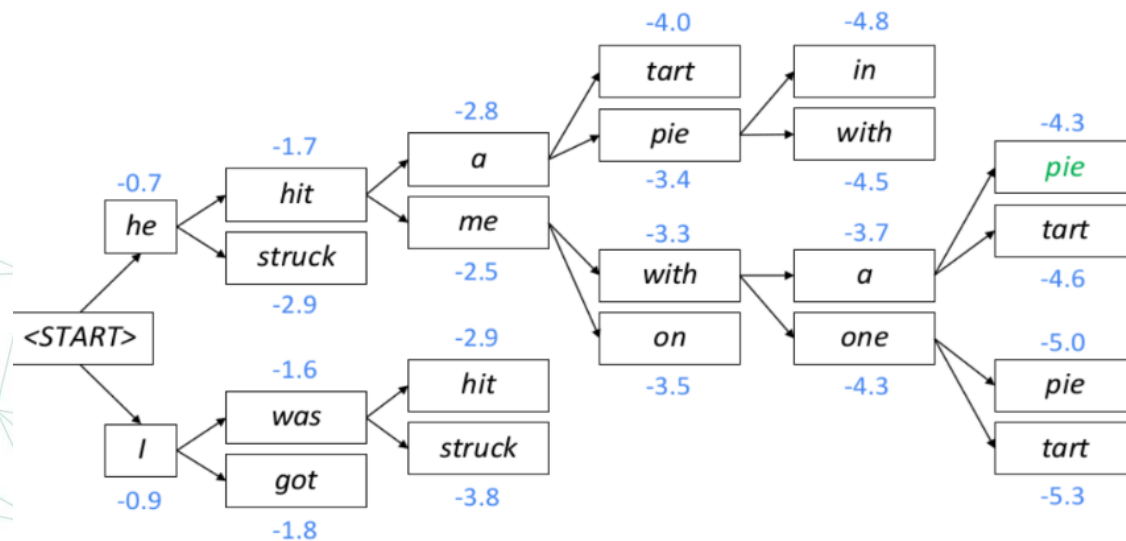


For each of the k hypotheses, find top k next words and calculate scores

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

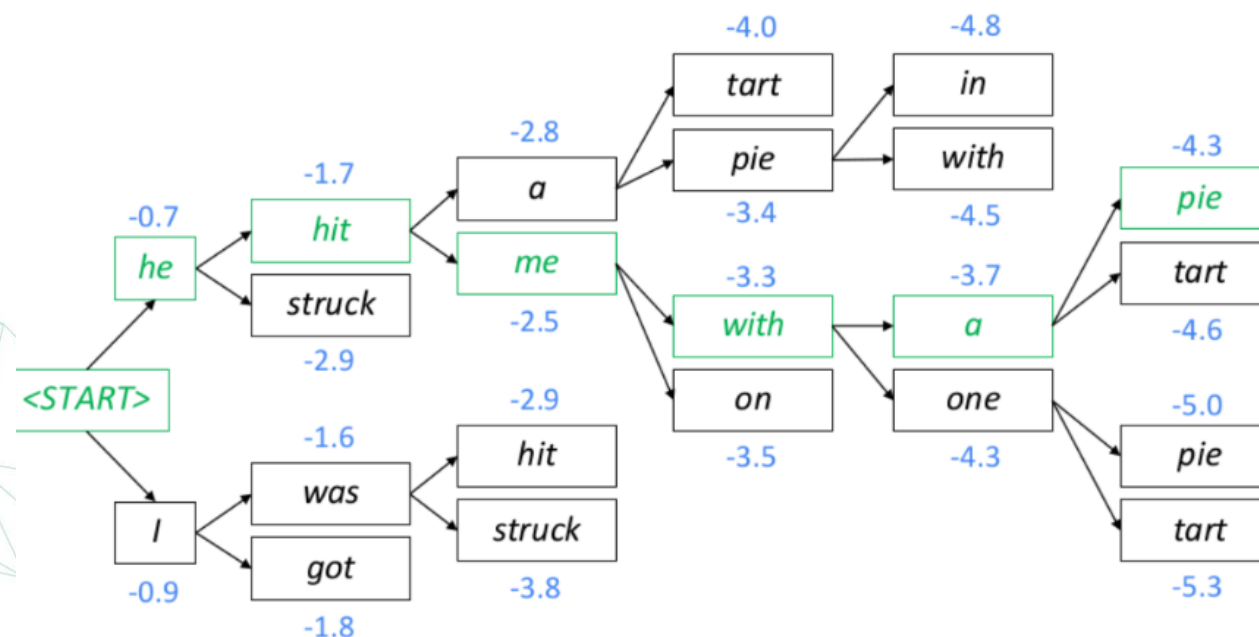


This is the top-scoring hypothesis!

Beam Search

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

Beam search decoding :stopping criterion

We have our list of completed hypotheses.
How to select top one with highest score?

Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Problem with this: longer hypotheses have lower scores

Fix: Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Evaluation with BLEU

- BLEU stands for **Bilingual Evaluation Understudy** and is a way to evaluate how good a translation is. Interestingly it comes from a [paper](#) from 2002, way before computers became good at translation.
- In essence, it measures **similarity** by counting **n-gram overlap** between the prediction and a reference translation. The formula is given by.

$$\text{BLEU} = \min \left(1, \exp \left(1 - \frac{\text{reference_len}}{\text{candidate_len}} \right) \right) \left(\prod_{i=1}^4 \text{precision}_i \right)^{\frac{1}{4}}$$

where

$$\text{precision}_i = \frac{\sum_{C \in \text{Candidates}} \sum_{i \in C} \min(n_{\text{cand_match}}^i, n_{\text{ref}}^i)}{\sum_{C \in \text{Candidates}} \sum_{i \in C} n_{\text{cand}}^i}$$

$$\text{BLEU} = \min \left(1, \exp \left(1 - \frac{\text{reference_len}}{\text{candidate_len}} \right) \right) \left(\prod_{i=1}^4 \text{precision}_i \right)^{\frac{1}{4}}$$

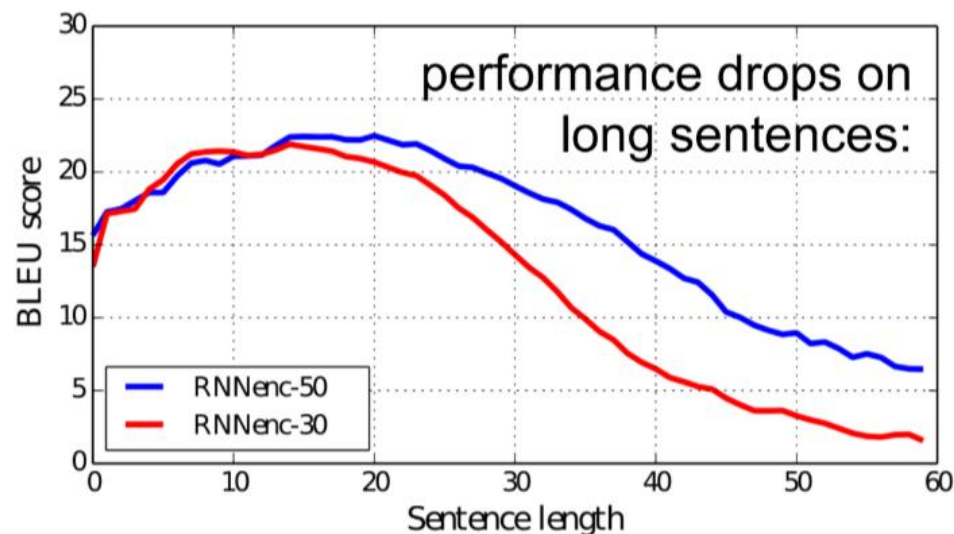
where

$$\text{precision}_i = \frac{\sum_{C \in \text{Candidates}} \sum_{i \in C} \min(n_{\text{cand_match}}^i, n_{\text{ref}}^i)}{\sum_{C \in \text{Candidates}} \sum_{i \in C} n_{\text{cand}}^i}$$

- The first term in the BLEU formula is a **brevity penalty**, which penalizes candidate translations that are short relative to the reference translation. The second term measures the **precision** of 1-gram, 2-grams, 3-grams and 4-grams by counting how many times they **overlaps** in the candidate and reference translations. The min is there to prevent predictions filled with the same word from getting high scores.
- Whilst BLEU is a very useful metric and still widely popular today, it does have a significant drawback. This is that it doesn't take **word meaning** into account. So if two sentences have similar meaning but use different words then BLEU won't recognize this. There are other metrics out there like ROUGE but evaluating translation is still an **active area of research**.

Bottleneck

- When sequence-to-sequence models were first being experimented with, they were found to perform impressively for short sequences but really struggle to translate **long sentences**. This is because the model in its current form has a significant **information bottleneck** between the encoder and decoder.
- All the information from the input sequence has to be **compressed** into a **single fixed length vector**, which is then used to produce a translation via the decoder. As the input sequence gets longer, there is **less space** to carry all of the complex language information that is needed to produce an accurate translation. This is where the **attention mechanism** comes in.

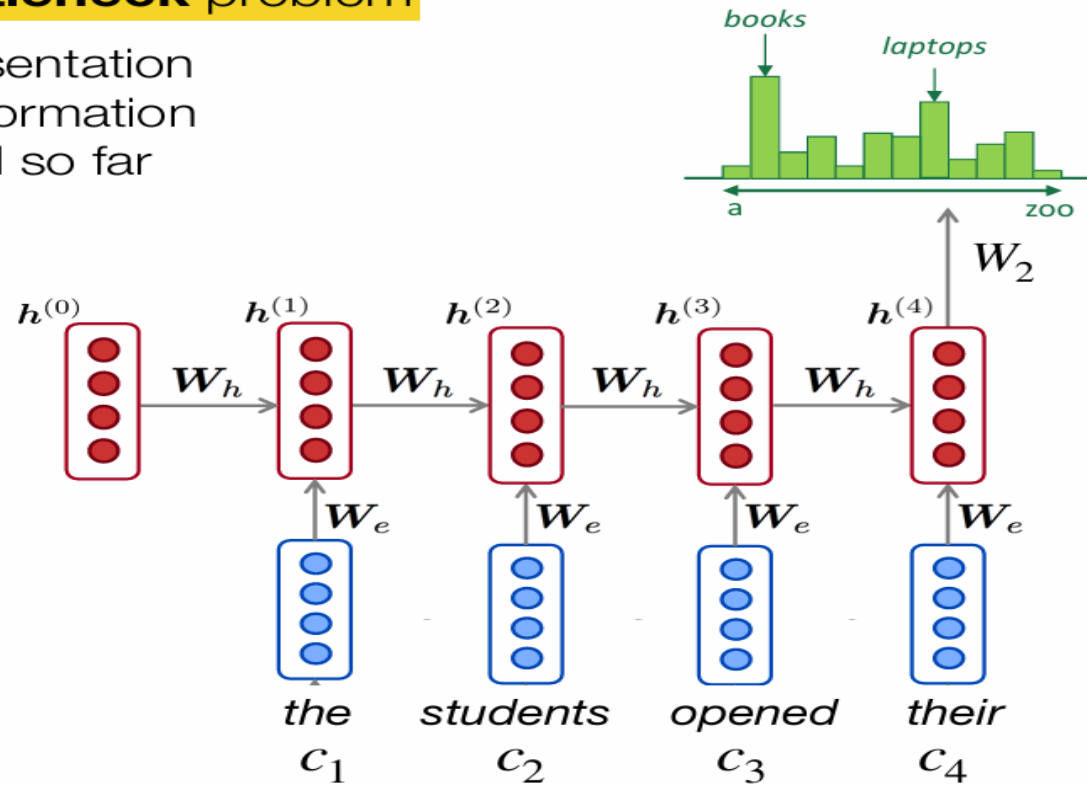


Bottleneck

RNNs suffer from a **bottleneck** problem

The current hidden representation must encode all of the information about the text observed so far

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$

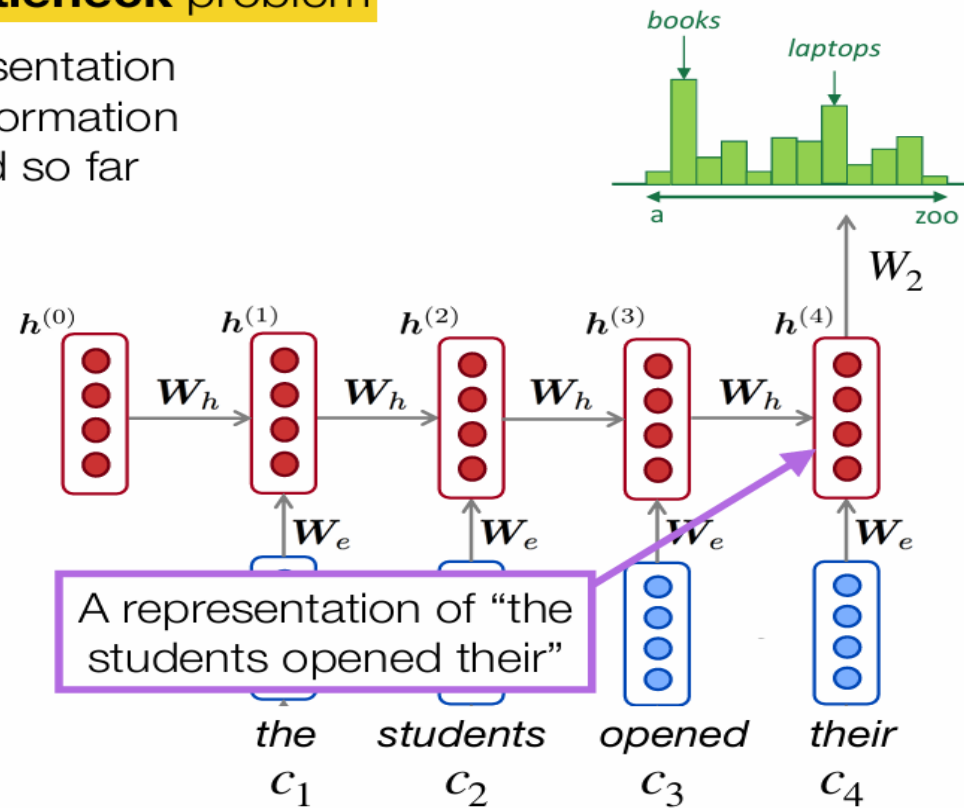


Bottleneck

RNNs suffer from a **bottleneck** problem

The current hidden representation must encode all of the information about the text observed so far

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$

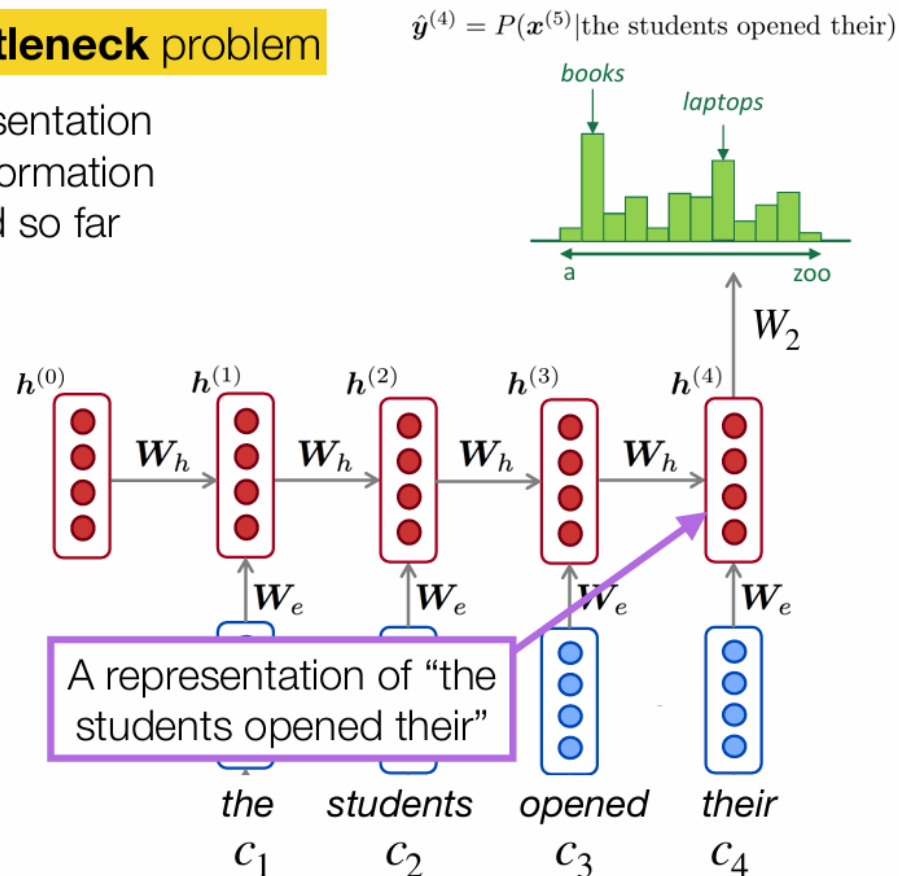


Bottleneck

RNNs suffer from a **bottleneck** problem

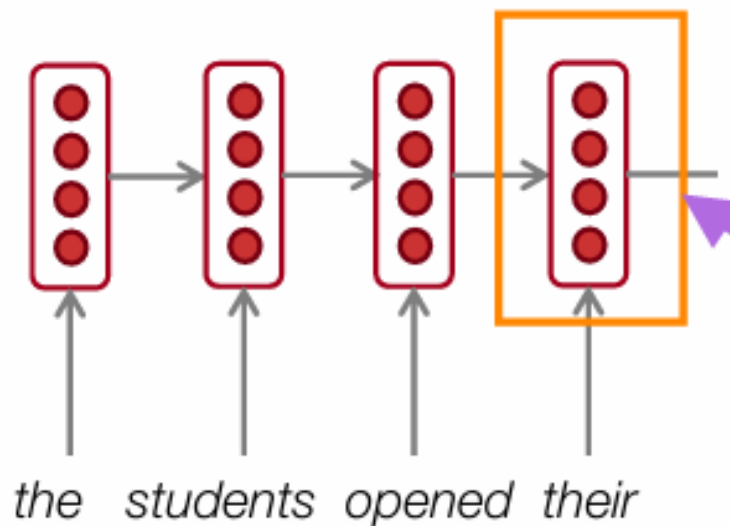
The current hidden representation must encode all of the information about the text observed so far

This becomes difficult especially with longer sequences



“you can’t cram the meaning of a whole %&@#&ing sentence into a single \$*(&@ing vector!”

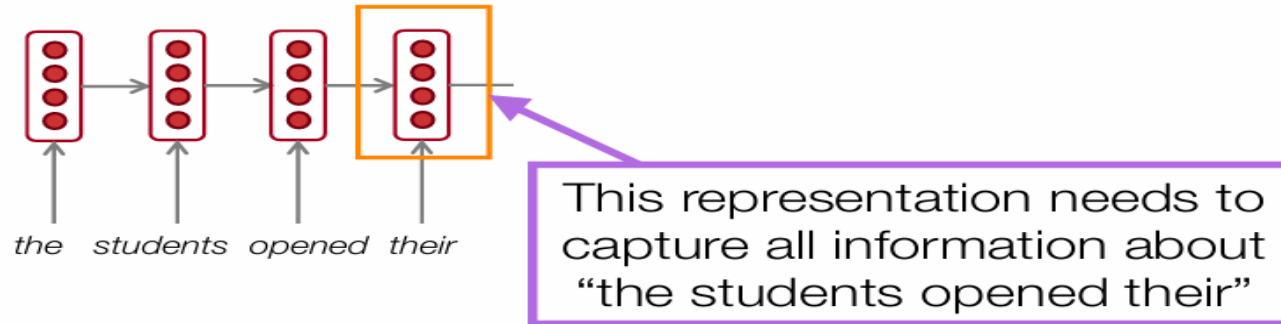
idea: what if we use multiple vectors?



This representation needs to capture all information about "the students opened their"

idea: what if we use multiple vectors?

idea: what if we use multiple vectors?



Instead of this, let's try:

the students opened their =  (all 4 hidden states!)

The solution: attention

Attention provides a solution to the bottleneck problem

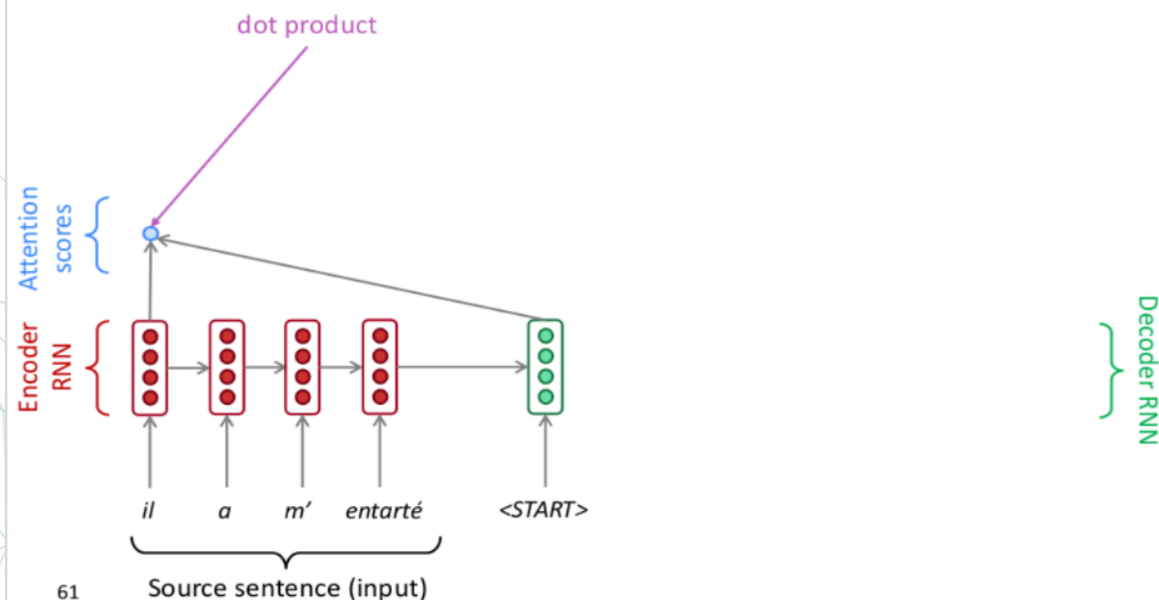
- **Attention mechanisms** (Bahdanau et al., 2015) allow language models to focus on a particular part of the observed context at each time step
- Originally developed for machine translation, and intuitively similar to word alignments between different languages

How does it work?

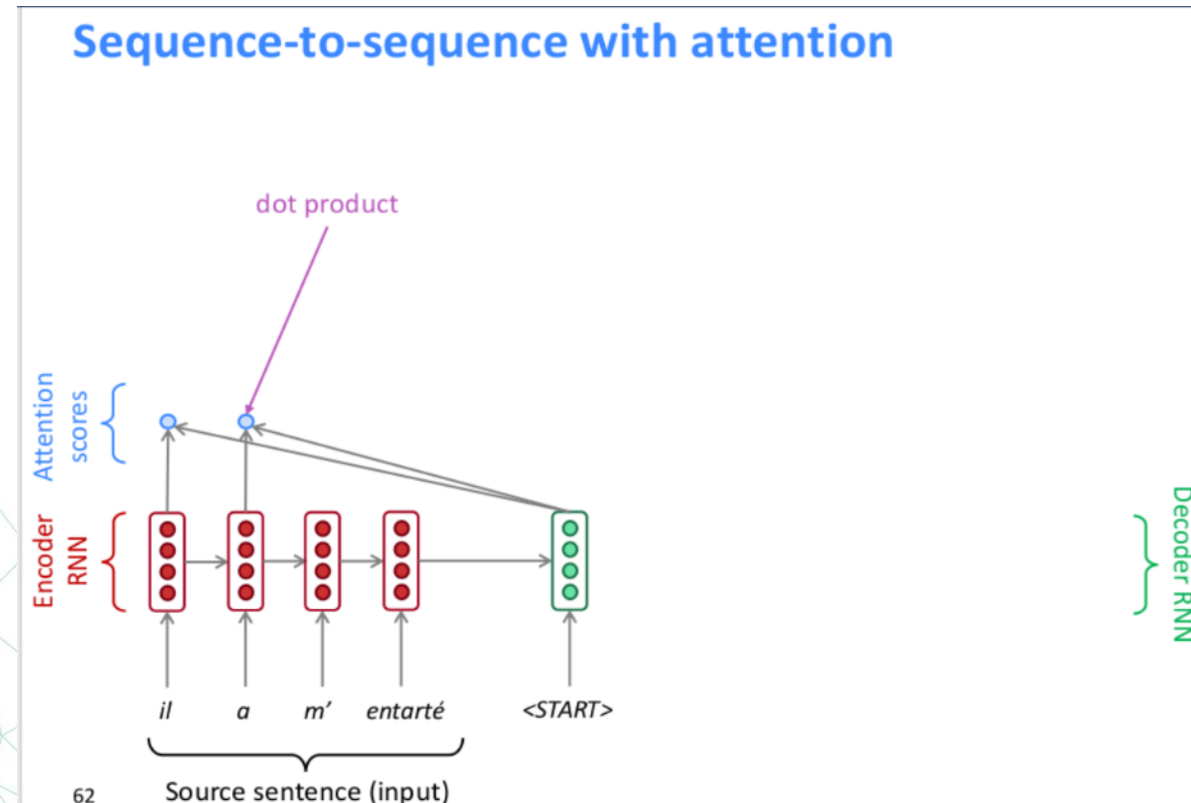
- in general, we have a single query vector and multiple key vectors. We want to score each query-key pair
- in a neural language model, what are the queries and keys?

Attention mechanisms in neural language models

Sequence-to-sequence with attention

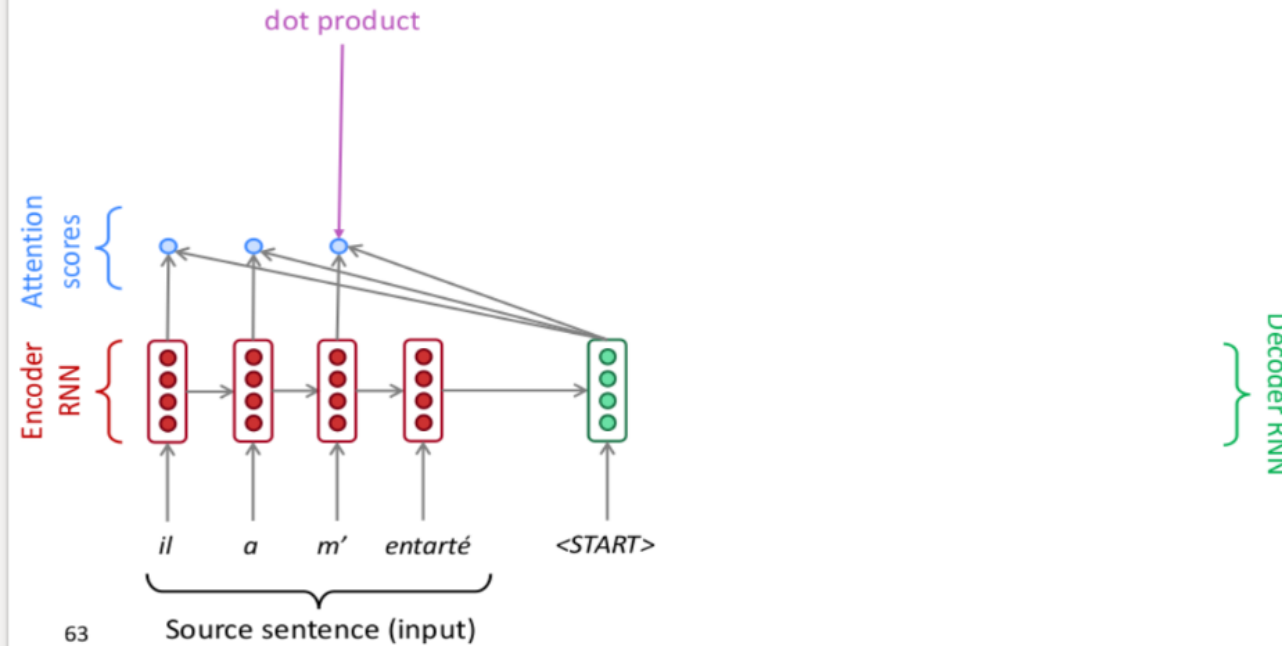


Attention mechanisms in neural language models



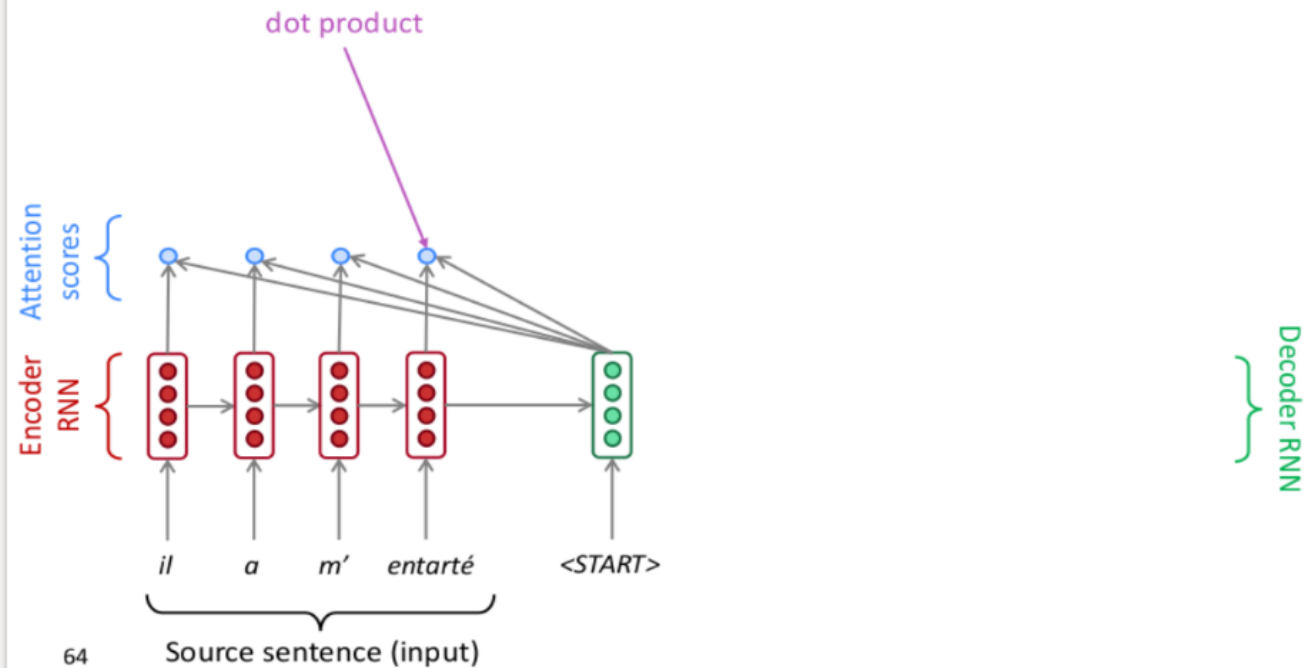
Attention mechanisms in neural language models

Sequence-to-sequence with attention

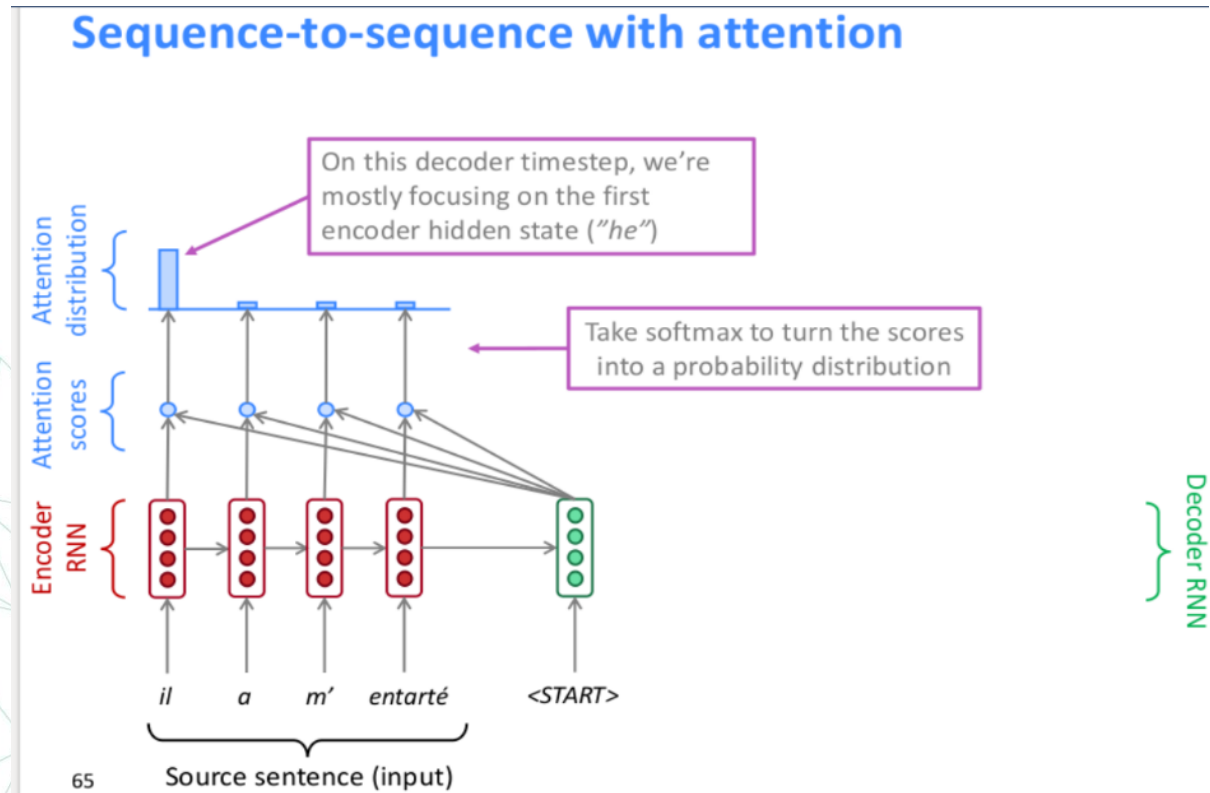


Attention mechanisms in neural language models

Sequence-to-sequence with attention

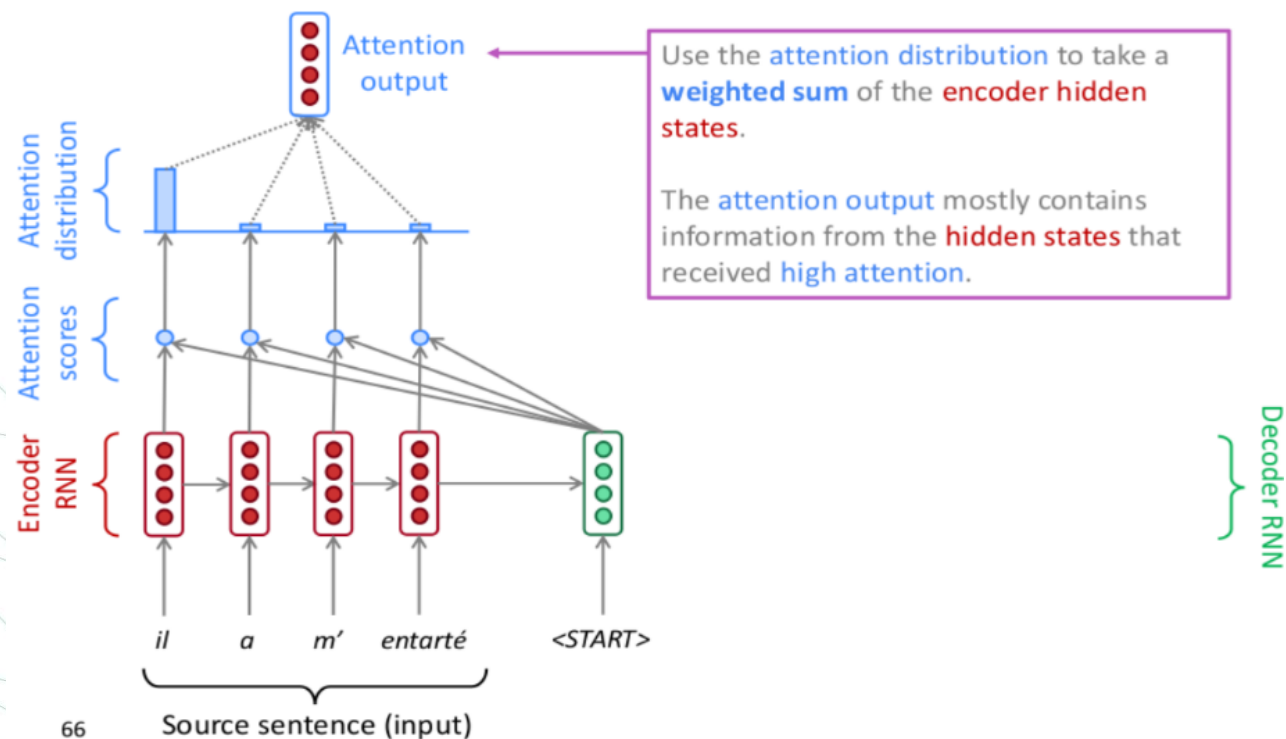


Attention mechanisms in neural language models



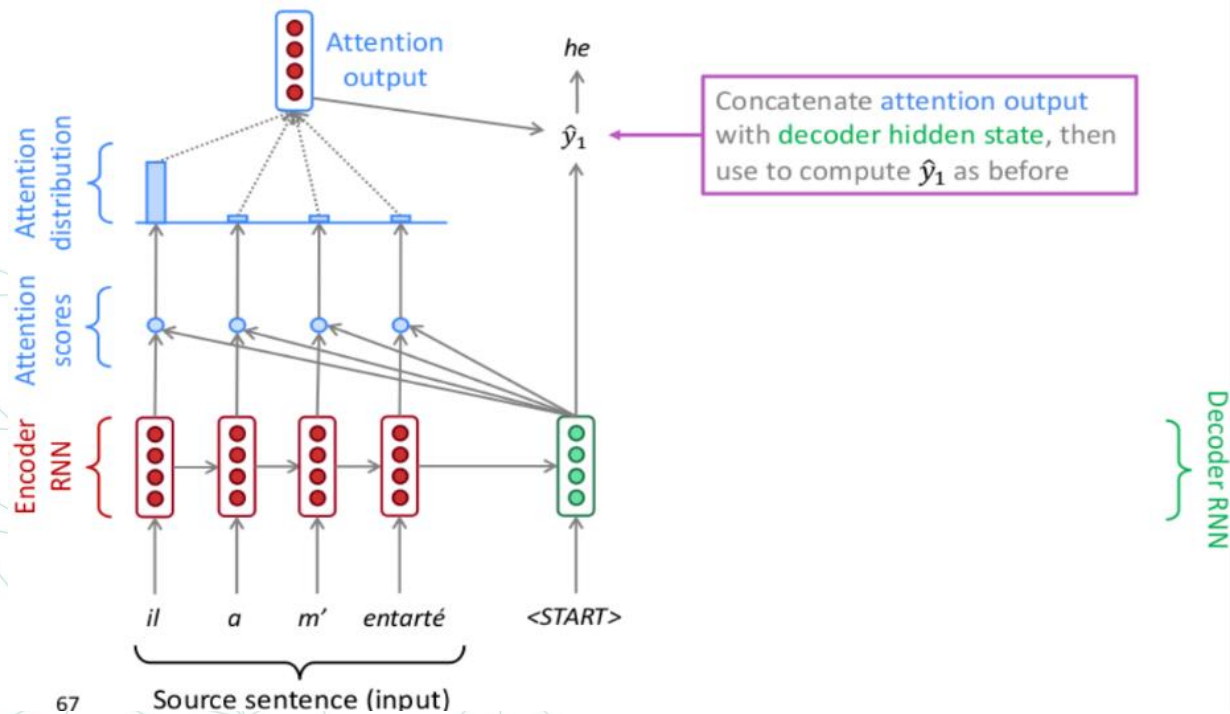
Sequence-to-sequence with attention

Sequence-to-sequence with attention



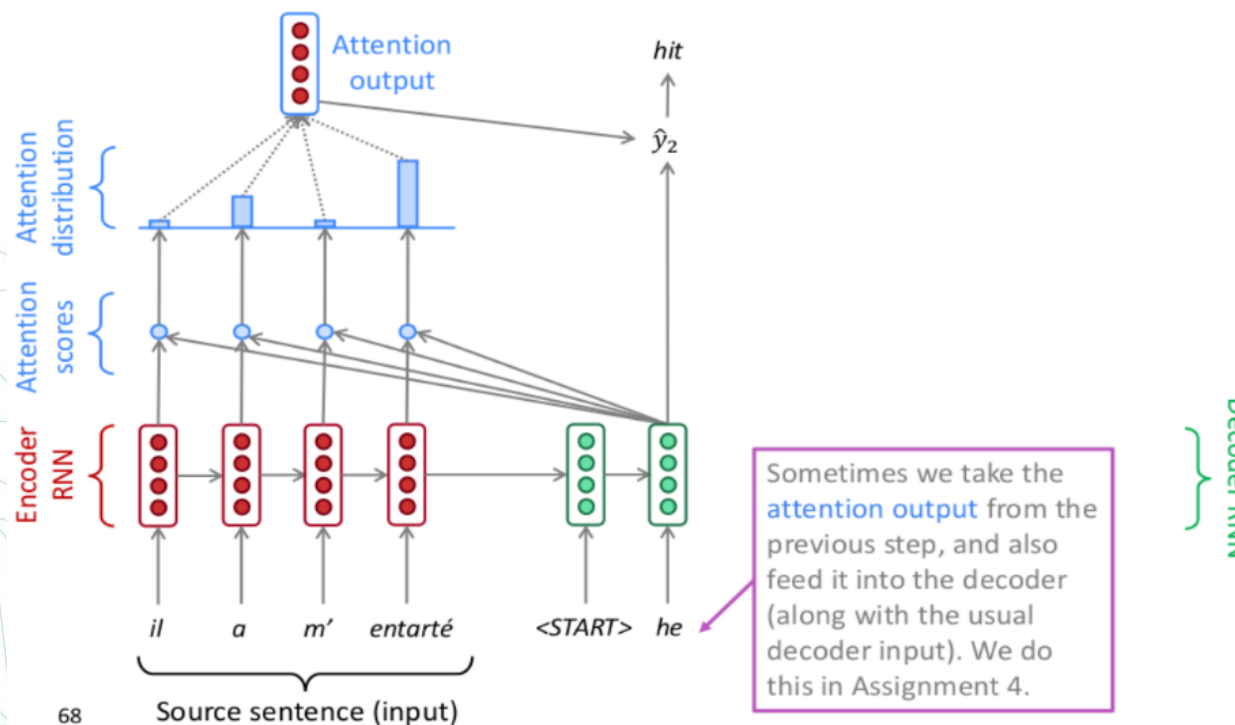
Sequence-to-sequence with attention

Sequence-to-sequence with attention

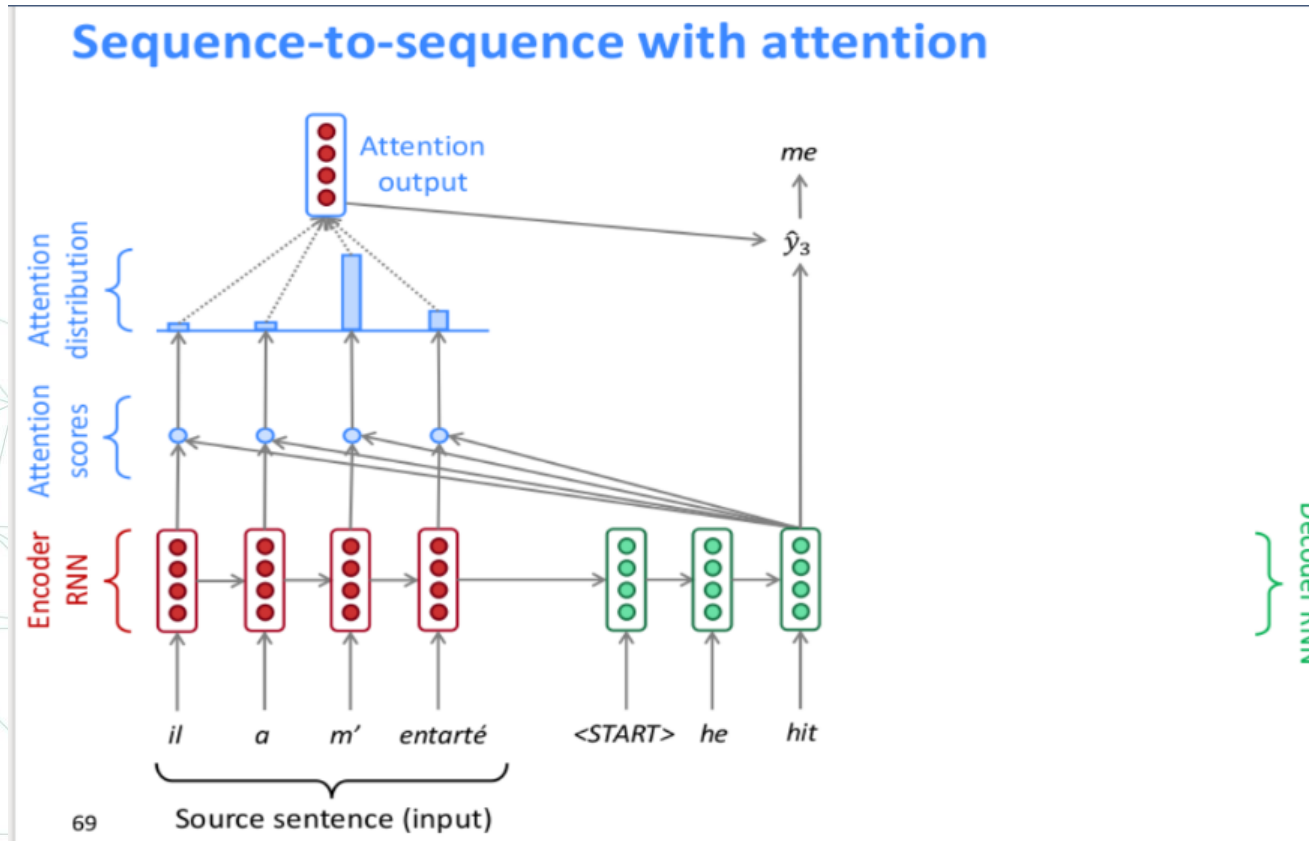


Sequence-to-sequence with attention

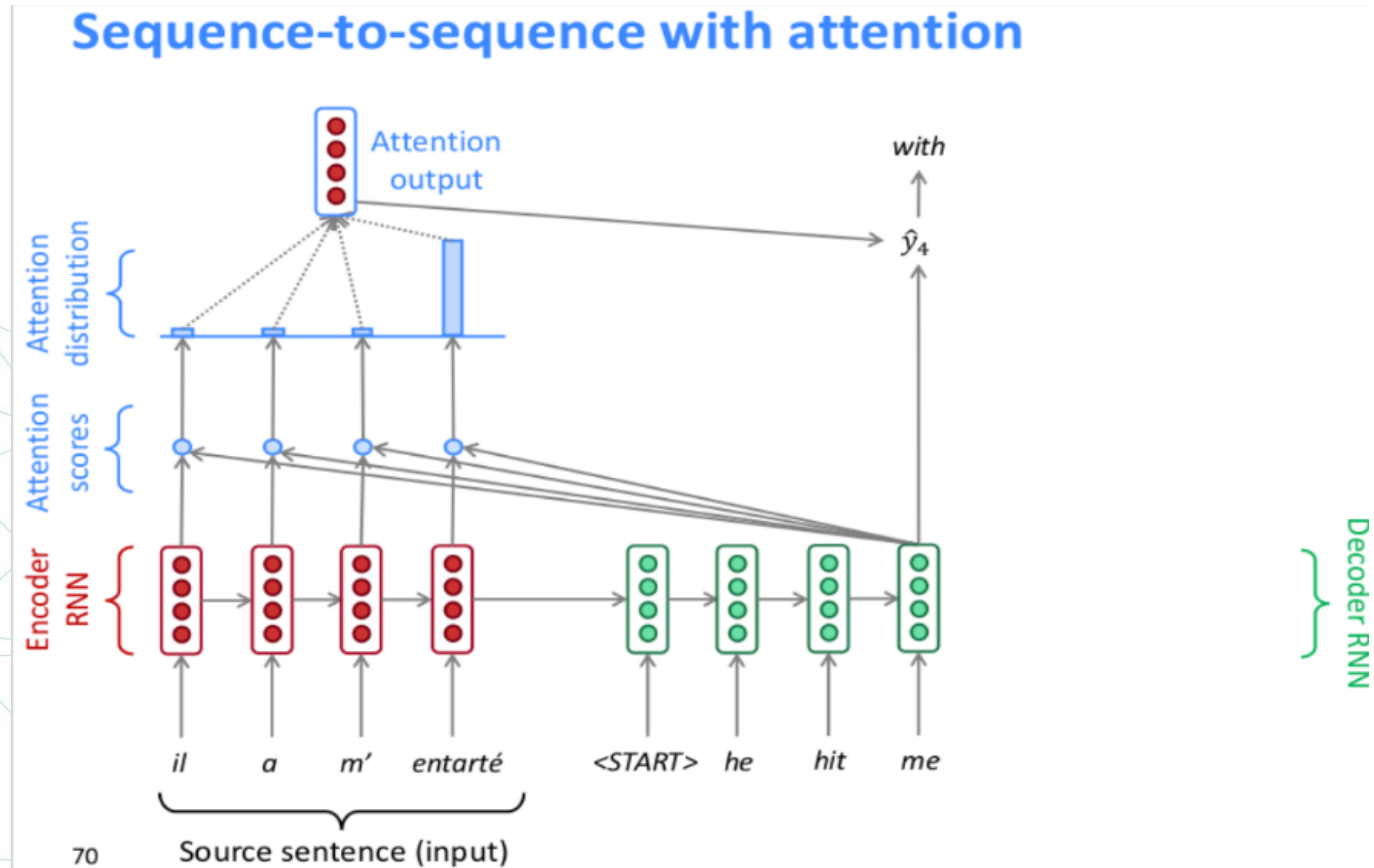
Sequence-to-sequence with attention



Sequence-to-sequence with attention



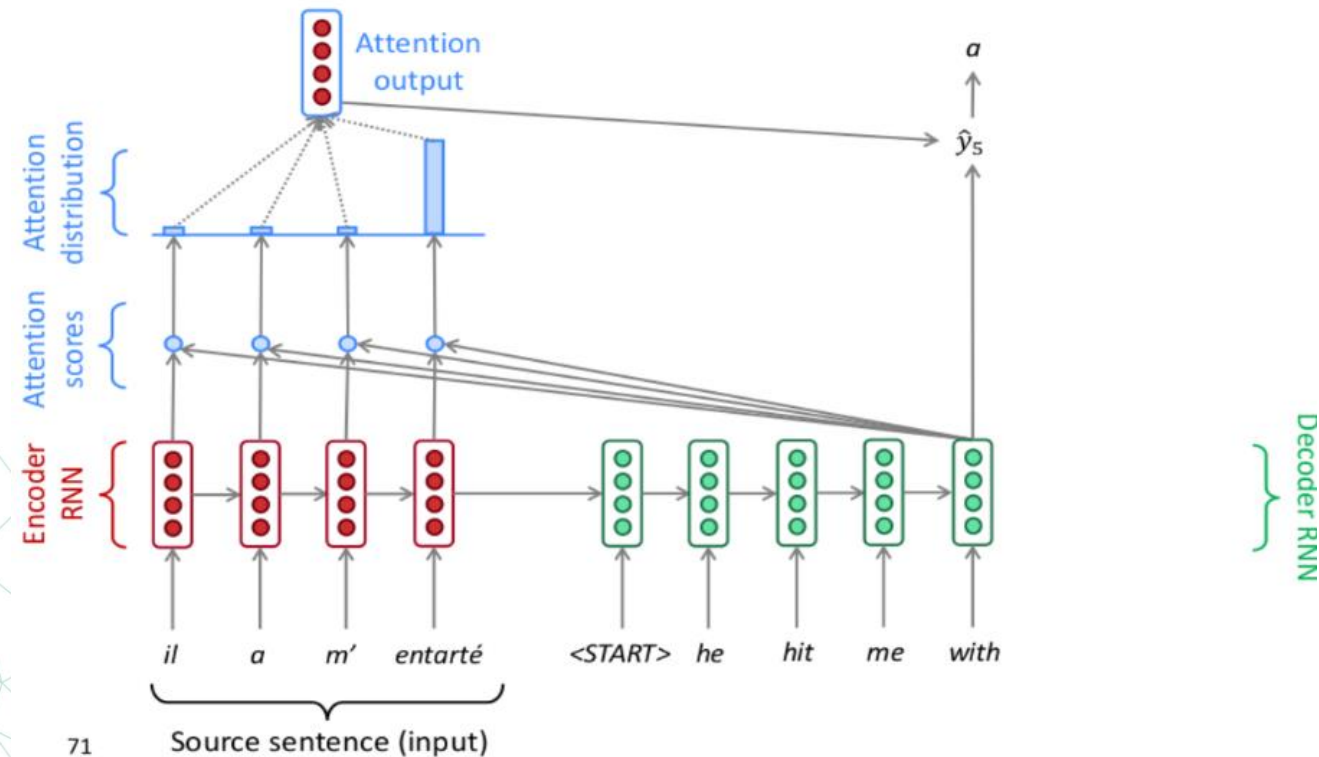
Sequence-to-sequence with attention



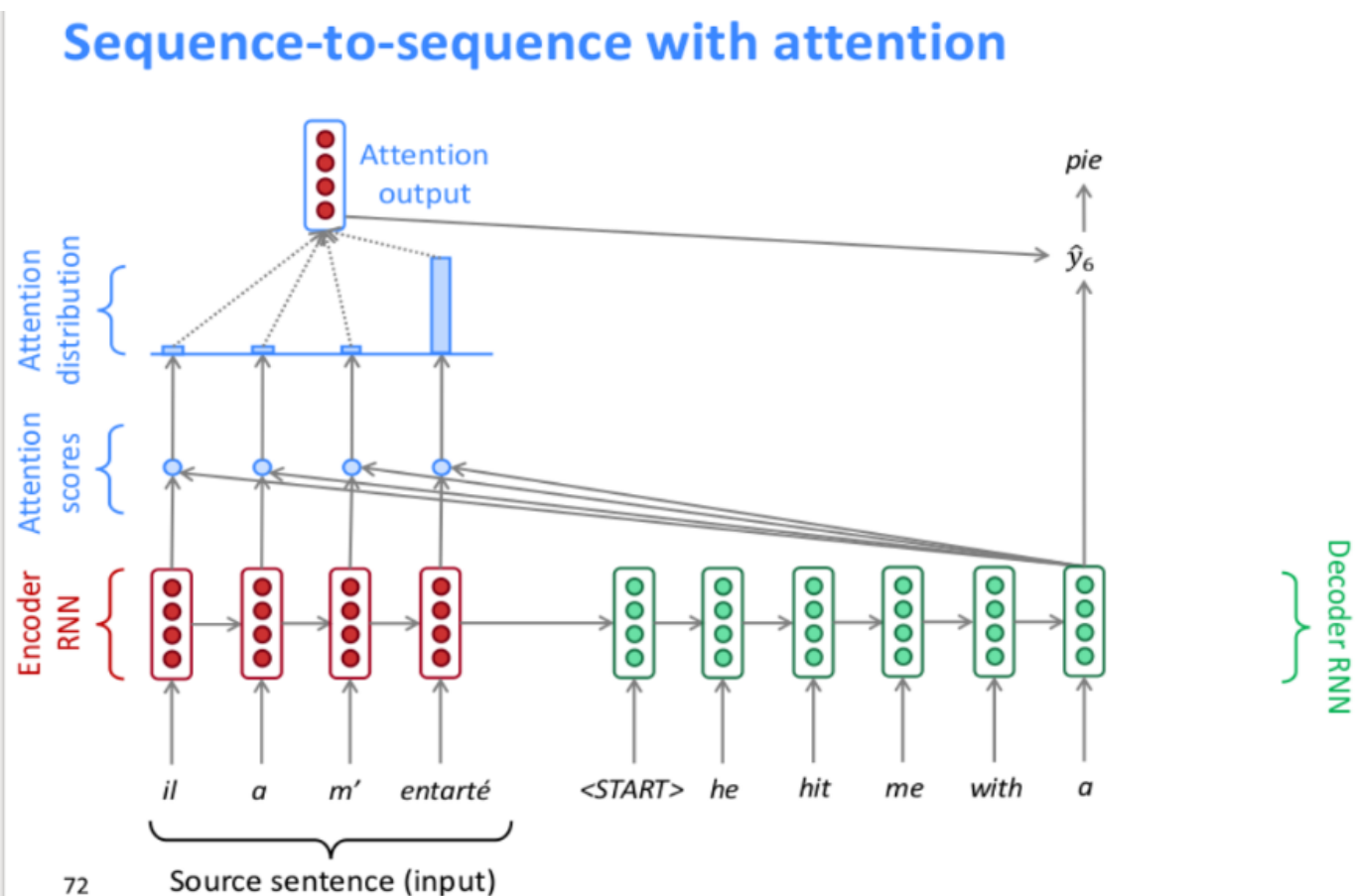
70

Sequence-to-sequence with attention

Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention

Attention: in equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[a_t; s_t] \in \mathbb{R}^{2h}$$

Sequence-to-sequence with attention

- Attention solves the bottleneck problem
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
 - Provides shortcut to faraway states
- Attention significantly improves NMT performance

Thanks